



Implementation and Characterization of Power, Performance and Area Impact of Triple Modular Redundancy for Post-Quantum Cryptography Hardware

handed in by

Fiderana Gerard Jacky

E-mail: f.jacky@campus.tu-berlin.de

Matriculation number: 483732

from Fakultät IV - Elektrotechnik und Informatik
der Technischen Universität Berlin
zur Erlangung des akademischen Grades

BACHELOR OF SCIENCE (B.Sc.)

Supervisors:

Reviewer: Prof. Dr. Jean-Pierre Seifert, Technische Universität Berlin
Reviewer: Prof. Dr.-Ing. Friedel Gerfers, Technische Universität Berlin
Supervisor: Lars Renkes, M. Sc., Technische Universität Berlin

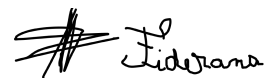
September 13, 2026

Berlin

Declaration

I hereby declare that I have completed this paper independently and on my own, without any unauthorized assistance from others, and using only the sources and resources listed. In preparing this thesis I used the large language model Claude (Anthropic) as a writing aid, for language correction and for polishing the text. All content, results, and conclusions are my own.

Berlin, September 13, 2026

A handwritten signature in black ink, appearing to read 'Fiderana', with a stylized, scribbled mark to its left.

Fiderana Gerard
Jacky

E-mail:

f.jacky@campus.tu-berlin.de

Matriculation number:

483732

Abstract

Post-quantum cryptographic hardware, such as implementations of CRYSTALS-Dilithium, is increasingly deployed in security-critical systems where fault attacks pose a realistic threat to key material and signature integrity. Triple Modular Redundancy (TMR) is a well-known countermeasure.

This study implements and characterizes TMR protection for a CRYSTALS-Dilithium hardware signing core on a Xilinx Artix-7 FPGA, with a Kintex-7 FPGA used as a secondary cross-device comparison. A Python-based netlist transformation replaces standard FDRE/FDSE flip-flops with TMR-protected equivalents (`trm_fdre`, `trm_fdse`), each exposing a fault-detection signal that feeds a sliding-window fault counter. Four protection strategies are evaluated against an unprotected baseline: full-design TMR, and three selective strategies protecting only the control state, only the output path, or the output path plus the control state. All five designs are functionally verified against Known-Answer-Test vectors and characterized through Vivado synthesis and implementation. Four of the five are also characterized on the Kintex-7 device for cross-device comparison.

Full TMR increases the flip-flop count by 200.6% and degrades the worst-negative-slack timing by up to 6.819 ns relative to the baseline, while selective protection achieves proportionally smaller overhead. SAIF-informed on-chip power on the Artix-7 device ranges from 0.326 W to 0.726 W across the five designs, and full protection does not fit on the Kintex-7 part at all.

A system-level fault-injection campaign confirms that protected registers mask injected faults inside real signing operations and that unprotected ones corrupt the signature or hang it outright. It also identifies two limits that survive full coverage: a common-mode fault defeats the majority vote, and storage that synthesis maps onto shift-register primitives rather than flip-flops lies outside the transformation's reach entirely.

Zusammenfassung

Post-Quanten-Kryptografie-Hardware, wie beispielsweise Implementierungen von CRYSTALS-Dilithium, kommt zunehmend in sicherheitskritischen Systemen zum Einsatz, in denen Fehlerangriffe eine reale Bedrohung für Schlüsselmaterial und die Integrität von Signaturen darstellen. Triple Modular Redundancy (TMR) ist eine etablierte Gegenmaßnahme gegen derartige Angriffe.

In dieser Arbeit wird ein TMR-Schutz für einen Hardware-Signaturkern von CRYSTALS-Dilithium auf einem Xilinx Artix-7 FPGA implementiert und charakterisiert. Zusätzlich wird ein Kintex-7 FPGA für einen geräteübergreifenden Vergleich herangezogen. Eine Python-basierte Netzlistentransformation ersetzt die standardmäßigen FDRE/FDSE-Flipflops durch TMR-geschützte Äquivalente (`trm_fdre`, `trm_fdse`). Jedes dieser Flipflops stellt ein Fehlererkennungssignal bereit, das einem Fehlerzähler mit Sliding Window zugeführt wird.

Vier Schutzstrategien werden mit einem ungeschützten Referenzdesign verglichen: vollständiger TMR-Schutz des gesamten Designs sowie drei selektive Strategien, bei denen ausschließlich der Steuerzustand, ausschließlich der Ausgabepfad oder sowohl der Ausgabepfad als auch der Steuerzustand geschützt werden. Alle fünf Designs werden anhand von Known-Answer-Test-Vektoren funktional verifiziert und mittels Vivado-Synthese und -Implementierung charakterisiert. Vier der fünf Designs werden zusätzlich auf dem Kintex-7 FPGA untersucht, um einen geräteübergreifenden Vergleich zu ermöglichen.

Vollständiges TMR erhöht die Anzahl der Flipflops gegenüber dem Referenzdesign um 200,6 % und verschlechtert das Worst-Negative-Slack-Timing um bis zu 6,819 ns, während die selektiven Schutzstrategien entsprechend geringere Overheads verursachen. Die SAIF-gestützte On-Chip-Leistungsaufnahme auf dem Artix-7 FPGA liegt für die fünf untersuchten Designs zwischen 0,326 W und 0,726 W; vollständiger Schutz passt auf den Kintex-7 gar nicht erst auf den Baustein.

Eine systemweite Fehlerinjektionskampagne bestätigt, dass geschützte Register injizierte Fehler innerhalb realer Signiervorgänge maskieren, während ungeschützte Register die Signatur verfälschen oder den Vorgang blockieren. Sie zeigt zudem zwei

Grenzen, die auch bei vollständigem Schutz bestehen bleiben: Ein Common-Mode-Fehler überwindet die Mehrheitsentscheidung, und Speicher, den die Synthese auf Schieberegister-Primitive statt auf Flipflops abbildet, bleibt von der Transformation vollständig unerreicht.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Lars Renkes, for his guidance and valuable feedback throughout this thesis. His advice and our discussions were invaluable in shaping both the direction and the quality of this work.

My thanks also go to Prof. Dr. Seifert and Prof. Dr.-Ing. Gerfers for taking the time to review and evaluate this thesis.

I am also grateful to the Chair of Security in Telecommunications at TU Berlin for providing the resources and technical environment necessary to carry out this study.

Finally, I am deeply grateful to my family and friends for their constant encouragement and support throughout my studies.

Contents

Declaration	iii
Abstract	v
Zusammenfassung	vii
Acknowledgements	ix
1 Introduction	1
1.1 Motivation and Problem Statement	1
1.2 Research Objective	2
1.3 Contributions	3
2 Preliminaries	5
2.1 Fault Attacks	5
2.2 Fault Countermeasures	6
2.2.1 Spatial Redundancy	6
2.2.2 Triple Modular Redundancy	6
2.3 CRYSTALS-Dilithium	7
2.3.1 Overview	7
2.3.2 Hardware-Relevant Structure of the Signing Operation	7
2.3.3 Security Levels	8
2.4 Power, Performance, and Area Criteria	9
2.4.1 Static Timing Analysis	9
2.5 Xilinx 7-Series FPGA Basics	10
2.5.1 Configurable Logic Blocks and Slices	10
2.5.2 Flip-Flop Primitives	11
2.5.3 Vivado Design Flow	11
2.6 Related Work	11
3 Design and Implementation	15
3.1 Threat Model and Design Goals	15
3.1.1 Threat Model	15

3.1.2	Design Goals	15
3.2	Triple Redundant Flip-Flop Design	16
3.2.1	FDRE	16
3.2.2	FDSE	18
3.3	Fault Counter Design	18
3.3.1	Chosen Design: OR-Reduced Sliding Window	18
3.3.2	Wiring <code>trigger_rst</code> to the Global Reset	20
3.4	Protection Strategies	21
3.4.1	Selection Rationale: Exposure Time	21
3.4.2	Full Protection	21
3.4.3	Output Path Only	22
3.4.4	Control State Only	22
3.4.5	Output Path Plus Control State	22
3.5	Automated Netlist Transformation	23
3.5.1	Starting Point: the Post-Synthesis Netlist	23
3.5.2	<code>generate_full_security.py</code> : Full-Design Transformation	23
3.5.3	Selective Transformation: a Module Allow-List	24
4	Verification and Evaluation	27
4.1	Functional Verification	27
4.1.1	Known-Answer-Test Methodology	27
4.1.2	Verification Results	27
4.1.3	A Verification Finding: the Fault Counter X-Propagation Bug	27
4.1.4	Unit-Level Fault Injection	28
4.2	Area, Performance, and Power Characterization	28
4.2.1	Area	28
4.2.2	Performance	31
4.2.3	Power	32
4.2.4	Throughput and Energy per Signature	33
4.3	Cross-Device Comparison	34
4.4	Fault-Rate Observability	36
5	Discussion	39
5.1	Research Question Revisited	39
5.2	Limitations	41
5.3	Future Work	42
6	Conclusion	45
	Bibliography	47

List of Figures	49
List of Tables	51
List of Listings	53
A Generator Script and Simulation Traces	55
A.1 Netlist Generation Script	55
A.2 TMR Flip-Flop Wrapper: trm_fdse	64
A.3 Simulation Trace Evidence	65
A.3.1 Fault Counter Threshold Logic (fault_counter_tb.v)	65
A.3.2 TMR Masking Guarantee, Traced Directly (based on fdre_tb.v)	66
A.3.3 Fault-Injection Testbench, Configuration and Force Mechanism	68
A.3.4 System-Level Fault Injection, Two Illustrative Cases	69

Introduction

1.1 Motivation and Problem Statement

The US National Institute of Standards and Technology (NIST) has standardized a descendant of CRYSTALS-Dilithium as ML-DSA in FIPS 204 [Nat24], and hardware implementations of Dilithium-family schemes are increasingly being deployed in embedded and security-critical systems, where signature generation must occur efficiently and under physical adversary access. In such settings, physical fault attacks, voltage glitching, clock glitching, and electromagnetic injection, among others, are realistic and practical threats. By deliberately corrupting a device's internal state at the right moment, an attacker can force faulty signatures or leak secret key material, independent of how mathematically sound the underlying signature scheme is. Both principal fault classes against this family of schemes have been demonstrated: a loop-abort fault that truncates the sampling of the random masking vector and strips it of its masking property, shown against BLISS [Esp+17], and a differential fault attack that exploits deterministic signing to compare a correct and a faulty signature, shown against Dilithium itself [BP18].

Triple Modular Redundancy is one well-established countermeasure against faults of this kind: because the vote resolves combinationally within the same clock cycle, it adds no latency to the operation it protects. What it costs instead is silicon. It triplicates every protected register outright and adds voter logic on every protected path, so it is paid for in area, in timing margin, and in switching power alike. Area is the hardest of those three limits because it is absolute, and on a design that already occupies a large fraction of the target device that cost can decide whether protection is deployable at all and not just how expensive it is. Applying Triple Modular Redundancy to every flip-flop in this study's reference design, for instance, produces a netlist that needs 109,940 Slice LUTs, more than the 101,400 available on the Kintex-7 device used later in this study for cross-device comparison; full protection simply does not fit.

A single fit-or-not verdict is the coarsest possible answer, and it is close to the only one available. Selective redundancy has an established line of work in the radiation-tolerance community, but that work selects and evaluates subsets by reliability rather

than by cost, and what has been studied for Dilithium specifically is error detection on individual arithmetic blocks and not redundancy applied to the signing core as a whole (Section 2.6). No power, performance, and area measurement of structural redundancy on this hardware has been reported, at full coverage or at anything short of it.

That gap matters most for selective protection, which is not one design point but a family of them: more of the design protected at a higher cost, or less of it at a lower cost. Without a measured cost attached to each choice, a designer has nothing to weigh them against. This study supplies that cost. What a given strategy buys in coverage against a real adversary is a separate question and one this study does not set out to answer.

1.2 Research Objective

This study is guided by the following research question:

What power, performance, and area impact does Triple Modular Redundancy, applied both fully and selectively, have on post-quantum cryptographic hardware?

This study answers that question on a concrete reference design: the CRYSTALS-Dilithium hardware signing core of Beckwith, Nguyen, and Gaj [BNG21], targeting Xilinx Artix-7 (xc7a200tsbv484-1) and, for a second device family, Kintex-7 (xc7k160tiffv676-2L). An automated netlist transformation applies Triple Modular Redundancy to this design at four granularities, from full protection of every flip-flop down to three selective strategies that each protect a different, deliberately chosen subset of the design's held output and control-flow registers. The question is answered in three stages: functional verification of every resulting variant against Known-Answer-Test vectors, FPGA synthesis and implementation of every variant that verifies correctly to obtain its area, timing, and power figures, and a fault-injection campaign that exercises the protection mechanism inside real signing operations.

This study works with the earlier NIST Round 3 version of Dilithium [Bai+21], not the finalized ML-DSA of FIPS 204 [Nat24]. Round 3's default signing mode is deterministic: a fixed key and message always take the same number of restart

iterations, and therefore the same number of clock cycles. Both the Known-Answer-Test verification and every cycle count reported in this study rely on that property. ML-DSA defaults to randomized signing instead, with deterministic signing now optional, precisely because that fixed, input-derived randomness is what the differential fault attack described above exploits [BP18].

1.3 Contributions

- An automated Python transformation that retrofits Triple Modular Redundancy onto an unmodified, synthesized gate-level netlist, replacing every flip-flop primitive with a fault-detecting, majority-voted equivalent and wiring all of them into a shared fault counter.
- Four protection strategies applied to a complete CRYSTALS-Dilithium signing hardware implementation at different granularities.
- Functional verification of all five resulting design variants against Known-Answer-Test vectors, together with unit-level fault-injection experiments confirming that the majority vote masks an injected fault and that the fault counter registers it.
- A power, performance, and area characterization of all variants from FPGA synthesis and implementation under identical timing constraints, replicated on a second device family to confirm the measured trend is not an artifact of one device.
- A system-level fault-injection campaign that forces stuck-at faults into real signing operations, confirming that protected registers mask the injected fault, that unprotected registers corrupt or hang the signing operation, and that the fault counter's reset mechanism triggers and clears correctly.
- Two structural blind spots the same campaign uncovers: a register outside every strategy's Triple Modular Redundancy coverage that remains vulnerable even under full protection, and a common-mode fault that corrupts two of three redundant copies simultaneously and defeats the majority vote regardless of coverage.

Structure of this thesis. Chapter 2 introduces fault attacks, Triple Modular Redundancy, the CRYSTALS-Dilithium signing operation, the power, performance, and area criteria used throughout, and the relevant FPGA background, closing with a review of related work. Chapter 3 presents the threat model, the triple-redundant flip-flop and fault-counter designs, the four protection strategies, and the automated netlist transformation that applies them. Chapter 4 reports functional verification, the area, performance, and power characterization on both device families, and the fault-injection results. Chapter 5 revisits the research question in light of those results and sets out this study’s limitations and future work. Chapter 6 closes with the overall cost gradient and what it means for a designer choosing a protection strategy.

Preliminaries

2.1 Fault Attacks

A *fault attack* deliberately disturbs a device's physical operation via techniques such as supply voltage glitching, clock glitching, electromagnetic pulse injections, or laser pulses to corrupt an internal signal or register at a chosen moment, without physically breaking the device open or mathematically defeating its cryptography. The deviation it induces in the intended computation may be transient or permanent. The attacker's goal is to observe how the resulting faulty output differs from the correct one and to use that difference to recover secret information.

Lattice-based signature schemes built on the Fiat-Shamir-with-Aborts paradigm, including CRYSTALS-Dilithium (Section 2.3), have been broken by two distinct fault classes. In a *loop-abort fault*, the attacker terminates the loop that samples the random masking vector y coefficient by coefficient, typically by faulting its counter or the branch that exits it. The remaining coefficients stay at their initial zero value, so y takes on an abnormally low degree and no longer masks the secret key; Espitau et al. [Esp+17] showed that a single such signature can suffice for full key recovery against BLISS, and that the attack extends to other schemes in the family. In a *differential fault attack*, Groot Bruinderink and Pessl [BP18] exploited Dilithium's deterministic signing to obtain a correct and a faulty signature over the same message, whose difference reveals a secret key element; there a random fault almost anywhere in the signing procedure suffices.

Both classes make the loop counters and finite-state machines that drive sampling and rejection as security-relevant as the datapath that carries the signature, which is why this study evaluates protecting a design's control state alongside its output path. Neither class is fully contained by doing so: the differential attack in particular remains available from faults outside any selectively protected region.

2.2 Fault Countermeasures

Hardware fault-tolerance techniques generally fall into three categories. *Spatial redundancy* uses extra hardware, *temporal redundancy* uses extra time by repeating a computation on the same hardware, and *information redundancy* uses extra data, adding parity or error-correcting codes to the data structures themselves so that the datapath need not be replicated, at the cost of encoder and checker logic. This thesis deals with spatial redundancy.

2.2.1 Spatial Redundancy

Spatial redundancy instantiates two or more copies of the hardware and compares their results within the same clock cycle. Its main advantage is that it adds no latency: a fault is masked or detected in the cycle in which it occurs, which matters for a signing operation where masking a fault should cost no time. Its cost is paid in area and, since more logic switches every cycle, in power.

2.2.2 Triple Modular Redundancy

Triple Modular Redundancy [Neu56] triplicates a circuit element and combines the three outputs with a majority voter so that any single copy that disagrees with the other two is masked:

$$Q = (Q_a \wedge Q_b) \vee (Q_a \wedge Q_c) \vee (Q_b \wedge Q_c) \quad (2.1)$$

Because the vote happens combinatorially within the same cycle, TMR adds no latency, at the cost of roughly tripling the area of every protected element plus the voter logic itself. This study applies TMR specifically to flip-flops, triplicating each register and voting on its output every cycle.

A textbook TMR voter only masks a fault. It does not report that one occurred. This study extends the basic scheme with an explicit *disagreement* signal alongside the voted output.

$$fault = (Q_a \oplus Q_b) \vee (Q_b \oplus Q_c) \quad (2.2)$$

which is asserted whenever any of the three copies disagrees, independent of which one is wrong or whether the vote itself still produces the correct value.

TMR has two well-known limitations: it assumes that the three copies fail independently, so it offers no guarantee against a common-mode fault that upsets all three simultaneously, and the voter itself is a single point of failure unless it, too, is protected.

2.3 CRYSTALS-Dilithium

2.3.1 Overview

CRYSTALS-Dilithium [Duc+18] is a lattice-based digital signature scheme built on the *Fiat-Shamir with Aborts* paradigm [Lyu09]. A signer commits to a random value, derives a challenge from that commitment (replacing an interactive verifier with a hash function, the Fiat-Shamir transform), computes a response, and *rejects and restarts* whenever the response would reveal information about the secret key. Security follows from the distribution of an accepted signature being independent of the secret key, at the cost of an unpredictable number of restart iterations.

That count is nevertheless reproducible. Round 3 Dilithium’s default signing mode expands the per-attempt randomness from a seed derived from the secret key and the message [Bai+21], so signing the same input twice always takes the same number of restarts, which is exactly the property that this study’s Known-Answer-Test verification and every reported clock-cycle count rely on. A descendant of Dilithium was later standardized by NIST as ML-DSA in FIPS 204 [Nat24], which flips this default to a randomized (“hedged”) mode, with deterministic signing now optional, precisely because that fixed, input-derived randomness is what the differential fault attack of Section 2.1 exploits [BP18]. The specification referenced throughout this chapter is Round 3 Dilithium [Bai+21], not FIPS 204.

2.3.2 Hardware-Relevant Structure of the Signing Operation

The hardware implementation this study builds on, from Beckwith, Nguyen, and Gaj [BNG21], implements key generation, signing, and verification in a single core, selected at runtime through a two-bit mode input. This study exercises and protects

its signing operation. The discussion here is deliberately kept free of heavy lattice-theoretic notation (see [Bai+21] for the full algorithmic specification this hardware implements). The following structure is what matters for understanding the design decisions in Chapter 3:

- **Polynomial arithmetic:** Dilithium’s operations are polynomial multiplications over a ring, computed efficiently using the Number-Theoretic Transform, a finite-field analogue of the Fast Fourier Transform built from butterfly units (visible in the synthesized design as repeated butterfly/butterfly2x2 sub-modules).
- **Hashing:** both the Fiat-Shamir challenge derivation and Dilithium’s pseudo-random expansion of public parameters are built on SHA-3/SHAKE, implemented via the Keccak permutation (the keccak_top sub-modules).
- **Sampling, rejection, and control state:** a control finite-state machine (combined_top) governs sampling the per-attempt randomness, computing the candidate response, checking whether it must be rejected, and either emitting the signature or restarting the loop. This is the logic that the fault classes of Section 2.1 target, and the logic that two of this study’s four protection strategies are designed to cover.
- **Hint generation and application:** Dilithium signatures include “hint” bits that let a verifier reconstruct a rounded value exactly. makehint produces them during signing; usehint applies them.

2.3.3 Security Levels

Dilithium is parameterized into multiple security levels, commonly labeled 2, 3, and 5 in increasing order of security and computational cost. This study’s functional verification uses security level 2 Known-Answer-Test vectors throughout, since the reference hardware exposes the security level as a runtime parameter within the same synthesized design rather than requiring a separate one. Section 5.2 discusses the resulting verification scope.

2.4 Power, Performance, and Area Criteria

Hardware designs are conventionally compared along three axes, jointly referred to as PPA: how fast the design can be clocked, the energy it dissipates per unit time, and the programmable-logic resources it occupies. This study characterizes every TMR variant against all three, since none of them is free. Triplicated logic costs area directly, voter logic on the register-to-register path can cost timing margin, and the extra logic and its switching activity both cost power.

Area is reported as Slice LUT and Slice Register utilization, introduced in Section 2.5.1.

Performance is reported as Worst Negative Slack and the equivalent maximum operating frequency it implies, introduced in Section 2.4.1.

Power is decomposed by Vivado's `report_power` into two components. *Static power* is dissipated by leakage current that flows even while the circuit is idle,

$$P_{\text{static}} = V \cdot I_{\text{leak}} \quad (2.3)$$

and depends on the design's area and the process technology rather than on what it is doing at any given moment. *Dynamic power* is dissipated by signal transitions charging and discharging capacitance,

$$P_{\text{dynamic}} \propto \alpha \cdot C \cdot V^2 \cdot f \quad (2.4)$$

where α is the switching activity factor, C the net and gate capacitance being switched, V the supply voltage, and f the clock frequency. It therefore scales with how much of the design actually toggles, not merely how much of it exists, which is why the estimate depends heavily on how α is obtained. Both components are read from the same post-implementation report used for area and timing (Section 2.5.3).

2.4.1 Static Timing Analysis

After placement and routing, Vivado's static timing analyzer checks every register-to-register path to determine whether a signal launched from one flip-flop reaches the next in time to be captured on the following clock edge, given the constrained

clock period. The margin, or shortfall, on the single worst path is reported as the *Worst Negative Slack* (WNS), so a negative WNS means at least one path needs more time than the constraint allows and the design does not meet timing at that frequency. The independent *hold* check verifies the opposite failure, a signal arriving *too early* and being captured a cycle too soon, and is reported as the *Worst Hold Slack* (WHS).

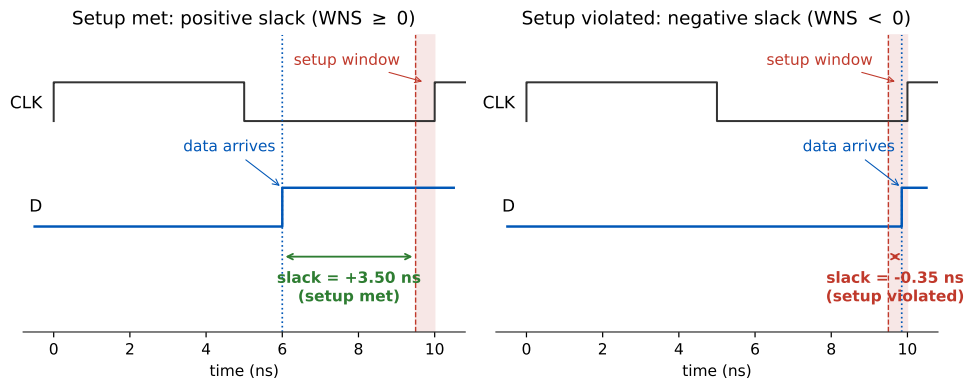


Fig. 2.1.: Setup-slack calculation for one register-to-register path.

Figure 2.1 shows the two cases side by side, differing only in the data-arrival time, which in the right panel is late enough to fall inside the setup window.

Because every path shares the same constrained period, the shortest period a design actually supports is $\text{period} - \text{WNS}$, which relaxes the constraint by $|\text{WNS}|$ when the slack is negative and tightens it when the slack is positive. This is how each variant's maximum operating frequency in Chapter 4 is obtained.

2.5 Xilinx 7-Series FPGA Basics

2.5.1 Configurable Logic Blocks and Slices

A Xilinx 7-Series FPGA's programmable logic is organized into Configurable Logic Blocks (CLBs), each containing two *slices*. Each slice provides four 6-input Look-Up Tables (LUTs), which implement arbitrary combinational logic of up to six inputs by directly storing their truth tables, and eight storage elements that can be configured as flip-flops or latches. This is why area is reported primarily as *Slice LUTs* (the combinational logic budget) and *Slice Registers* (the storage budget). They are physically distinct resources within the same slice, which is why a design's LUT and flip-flop utilization can grow at very different rates.

2.5.2 Flip-Flop Primitives

Xilinx synthesis tools map every HDL register onto one of four dedicated flip-flop primitives, all with a clock enable and distinguished by whether their control input is a set or a reset and whether it is synchronous or asynchronous: FDRE (synchronous reset), FDSE (synchronous set), FDCE (asynchronous clear), and FDPE (asynchronous preset). The reference Dilithium design uses only FDRE and FDSE instances, 29,098 of them across the signing datapath and control logic.

2.5.3 Vivado Design Flow

Turning the HDL source into a working bitstream proceeds through synthesis, implementation, and bitstream generation, the last of which is not used here since no design variant is programmed onto physical hardware. Only implementation, not synthesis alone, has enough information to produce a meaningful timing or power report. This is why every result in this study is drawn specifically from post-implementation (“routed”) reports rather than post-synthesis estimates.

2.6 Related Work

Applying redundancy selectively is not a new idea. It has an established line of work in the radiation-tolerance community, where the motivating fault is a randomly occurring single-event upset. Samudrala et al. [SRK04] identify SEU-sensitive subcircuits from signal probabilities and triplicate only those, reducing area overhead substantially relative to full TMR while retaining most of its protection. Pratt et al. [Pra+06] classify circuit structures by importance and prioritize those producing *persistent* errors, meaning state that does not correct itself once corrupted, reporting that selective mitigation can yield more reliability per unit cost than full mitigation. Keller and Wirthlin [KW21] continue this line for modern SRAM-based devices.

That persistence criterion is close to the exposure-time rationale used in this work, arrived at independently here from the observation that pipeline registers are overwritten every cycle while control state and held output registers are not. The difference lies in what the selection optimizes for and in what is then measured. Those works choose the subset that maximizes reliability per unit area under a random upset model, and they evaluate the result as reliability, through neutron cross

sections or fault-injection campaigns. This study instead chooses the subset that matters to a deliberate fault-attack adversary, protecting the control state those attacks target (Section 2.1), and evaluates the result as measured power, performance, and area.

TMR’s common-mode assumption has also been studied directly. Cannon et al. [Can+19] show that redundant copies placed and routed close together on an SRAM FPGA share failure modes, and that incremental placement, incremental routing, and striping can reduce the resulting sensitive neutron cross section, with their best technique reporting a $400\times$ reduction. Those mitigations act on physical placement, which the netlist-level transformation used here does not control, so they are complementary to rather than competing with this work.

On the hardware side, several unprotected Dilithium implementations exist. The reference design used throughout this study [BNG21] targets high throughput, and Land et al. [LSG22] present Artix-7 implementations covering all Round-3 parameter sets. Both optimize area and performance, and neither addresses fault tolerance.

Countermeasures for lattice signatures have so far been proposed mainly at the algorithmic level. Espitau et al. [Esp+17] suggest shuffling coefficient sampling and rejecting low-degree masking polynomials, but Ulitzsch et al. [Uli+23] later recover the secret key even with those countermeasures in place, by treating the faulty signatures as a noisy linear system and solving it with integer linear programming, and conclude that adequate protection likely requires substantially more expensive measures. More recently, Chowdhury et al. [Cho+25] propose a unified Kyber and Dilithium FPGA architecture whose fault tolerance rests on a probabilistic verification mechanism built on rejection sampling. These defenses operate on the algorithm. The approach characterized here is structural and algorithm-agnostic, applied to a synthesized netlist after the fact, and the two are in principle composable.

A related but distinct line targets the arithmetic itself rather than the algorithm’s control flow. Abdelmonem et al. [Abd+25] derive algebraic error-detecting vectors for the Number-Theoretic Transform used in Kyber, Dilithium, and Falcon, including the first method that detects up to two maliciously placed faults for Dilithium specifically, without full recomputation. Work in this line, whether its cost is reported algorithmically or benchmarked on an FPGA, covers one arithmetic block rather than the signing core as a whole.

Each strand above measures something different from what this study reports: reliability per unit area under a random-upset model, neutron cross section under given placement and routing choices, resistance to a specific algorithmic attack,

or the cost of detecting faults in a single arithmetic block. None reports power, performance, and area for a deliberately targeted, structurally selective redundancy scheme applied to a post-quantum signature core as a whole, per protection strategy and replicated on a second device family, which is what this study contributes.

Design and Implementation

3.1 Threat Model and Design Goals

3.1.1 Threat Model

This study assumes an attacker in physical possession of the device, able to inject a fault at a moment of their choosing and to repeat the attempt. The attacker is assumed to reach a register of their choosing, which is deliberately conservative, since it grants more precision than a realistic glitching setup provides. Their aim is an exploitable output, a forged or key-leaking signature of the kind the loop-abort and differential fault attacks of Section 2.1 produce. A device that merely stops emitting signatures is a failure but not a break.

Within that model, the mechanism developed in this chapter addresses transient faults disturbing a single redundant copy of a protected flip-flop, together with elevated rates of such faults. Four things lie outside it. A fault affecting two or more copies of the same register simultaneously defeats majority voting by construction, which the fault-injection campaign in Chapter 4 demonstrates directly. Storage that synthesis maps onto primitives other than flip-flops is never reached by the transformation, which the same campaign also demonstrates. The majority voter is itself not triplicated, and faults in combinational logic matter only insofar as their effect is latched; neither of those two is tested here. Side-channel and invasive attacks are outside the scope of this work entirely.

3.1.2 Design Goals

Four properties guided every design decision here:

- **No latency overhead:** the majority vote must resolve combinational within the same clock cycle in which a fault occurs, consistent with the spatial-redundancy choice.

- **Automation over manual modification:** with 29,098 flip-flops in the reference design, manually rewriting instances is impractical and error-prone. Every design variant is produced by a Python script operating directly on the post-synthesis netlist.
- **Configurable protection granularity:** the same underlying mechanism, TMR flip-flops feeding a fault counter, needs to support both full-design protection and three selective strategies without maintaining four separate hand-written netlists.
- **Fault observability and actionable response:** masking a fault is not sufficient on its own. An elevated fault rate must be observable and capable of triggering a corrective reset, since a device under active fault attack can be glitched repeatedly.

3.2 Triple Redundant Flip-Flop Design

Every Xilinx flip-flop primitive that the reference design uses is replaced by a wrapper module that triplicates it internally and exposes the same port interface as the original primitive. The replacement is a drop-in substitution at the netlist level, and no other part of the design needs to change to accommodate it.

3.2.1 FDRE

`trm_fdre`¹ wraps three internal FDRE instances (synchronous reset, clock enable):

```

1  module trm_fdre #(
2      parameter INIT = 1'b0
3  ) (
4      output wire Q,
5      output wire fault,
6      input wire C,
7      input wire CE,
8      input wire D,
9      input wire R
10 ) ;
11
```

¹The `trm_` prefix is a transposition of the TMR acronym that entered the codebase early and was kept for consistency across the module names, testbenches, and generated netlists that already referenced it. It carries no meaning beyond “TMR wrapper”.

```

12 (* KEEP = "true" *) wire q_a, q_b, q_c;
13
14 (* DONT_TOUCH = "true" *)
15 FDRE #(.INIT(INIT)) u_ff_a (.Q(q_a), .C(C), .CE(CE), .D(D), .
    R(R));
16 (* DONT_TOUCH = "true" *)
17 FDRE #(.INIT(INIT)) u_ff_b (.Q(q_b), .C(C), .CE(CE), .D(D), .
    R(R));
18 (* DONT_TOUCH = "true" *)
19 FDRE #(.INIT(INIT)) u_ff_c (.Q(q_c), .C(C), .CE(CE), .D(D), .
    R(R));
20
21 function vote3;
22     input a, b, c;
23     begin
24         vote3 = (a & b) | (a & c) | (b & c);
25     end
26 endfunction
27
28 (* KEEP = "true" *) wire q_voted = vote3(q_a, q_b, q_c);
29 assign Q      = q_voted;
30 assign fault = (q_a ^ q_b) | (q_b ^ q_c);
31
32 endmodule

```

Listing 3.1: TMR wrapper for FDRE.

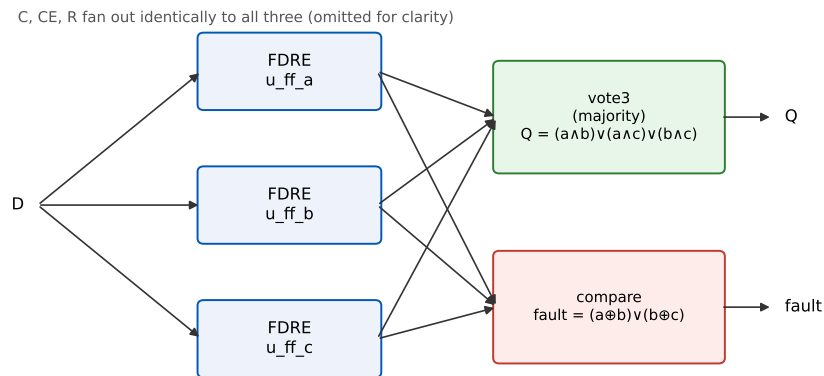


Fig. 3.1.: Block diagram of trm_fdre (Listing 3.1). C, CE, and R fan out identically to all three and are omitted for clarity.

Figure 3.1 lays out the same structure as Listing 3.1 visually. A single input fans out to three independent copies, and their three outputs feed two separate combinational

blocks, with the majority vote producing Q and the pairwise comparison producing fault.

Two synthesis attributes protect the triplication from the optimizer:

- (* KEEP = "true" *) on q_a/q_b/q_c. The three flip-flops are driven by identical inputs, so Vivado would otherwise recognize their outputs as equivalent and collapse them into a single net, silently undoing the triplication before the vote and comparison logic ever runs.
- (* DONT_TOUCH = "true" *) on each of the three FDRE instances, enforcing the same guarantee at the cell level by preventing the synthesizer from merging the instances themselves as redundant copies of the same logic.

3.2.2 FDSE

trm_fdse is structurally identical to trm_fdre, with a synchronous *set* in place of a synchronous *reset*, and is not reproduced here in full for that reason. The same (* KEEP *)/(* DONT_TOUCH *) reasoning applies verbatim. The reference netlist instantiates only FDRE and FDSE, so no wrapper is needed for the asynchronous FDCE and FDPE primitives.

3.3 Fault Counter Design

3.3.1 Chosen Design: OR-Reduced Sliding Window

Every trm_fdre or trm_fdse instance in a module exposes its own fault signal. These are OR-reduced, together with any child module's own aggregated signal, into a single module_fault line per module. Every module's module_fault line is threaded up through the instantiation hierarchy this way until it reaches combined_top, where all of them are OR-reduced once more into the one signal that drives the design's single fault_counter instance:

```
1  module fault_counter #(
2      parameter WIDTH    = 32,
3      parameter THR_100  = 10
4  ) (
5      input  wire          clk,
6      input  wire          rst,
```

```

7      input  wire          fault ,
8      output reg  [WIDTH-1:0] count ,
9      output reg          trigger_rst
10 );
11
12 wire fault_s = (fault === 1'b1);
13
14 reg window [0:99];
15 reg [WIDTH-1:0] running_total;
16 integer j;
17
18 initial begin
19     count = 0; running_total = 0; trigger_rst = 0;
20     for (j = 0; j < 100; j = j + 1) window[j] = 0;
21 end
22
23 always @(posedge clk) begin
24     if (rst) begin
25         count <= 0; running_total <= 0; trigger_rst <= 0;
26         for (j = 0; j < 100; j = j + 1) window[j] <= 0;
27     end else begin
28         if (fault_s) count <= count + 1'b1;
29         running_total <= running_total + fault_s - window
30             [99];
31         for (j = 99; j > 0; j = j - 1) window[j] <= window[j
32             -1];
33         window[0] <= fault_s;
34         trigger_rst <= (running_total + fault_s - window[99]
35             >= THR_100);
36     end
37 end
38 endmodule

```

Listing 3.2: OR-reduced sliding-window fault counter.

count accumulates every cycle in which at least one protected flip-flop anywhere in the design disagreed, counting from device power-up. The rst input is hardwired low in every generated netlist, so the counter is never reset during operation.

The fault_s wire treats an unknown fault input, for instance from a flip-flop that has not yet been written since power-up, as “no fault” rather than letting the unknown propagate indefinitely through the counter’s arithmetic. Case equality is a simulation-only construct, so this guard acts on the gate-level simulation model

rather than on the implemented hardware, which never encounters an unknown value. Chapter 4 describes the failure it prevents.

`trigger_rst` tracks a 100-cycle sliding window and asserts once the number of faulty cycles within that window reaches `THR_100` (10 by default), distinguishing an isolated, expected event from a sustained fault campaign. `running_total` is maintained as a running sum equal to the number of ones currently held in the window. Whenever `window[99]` is one, the window holds at least one fault, and `running_total` is therefore at least one, so the subtraction cannot underflow.

Both the window size and `THR_100` are synthesis-time parameters, and the appropriate values are deployment-specific. They depend on the expected fault rate of the operating environment and on the length of a signing operation, since a window much shorter than the operation gives the rate estimate little statistical basis while a much longer one delays detection. `THR_100 = 10` over a 100-cycle window is this study's demonstration default, applied identically to every variant.

Because the counter OR-reduces every flip-flop's fault signal before counting, it discards per-cycle fault *multiplicity*. A lone single-bit upset and several flip-flops faulting in the very same cycle both register as exactly one faulty cycle in the sliding window. An earlier design iteration preserved that information with a per-cycle popcount instead of an OR-reduction, but it was not used for any result here. Section 5.2 and Section 5.3 discuss that alternative and the multibit-fault detection gap it would have closed.

3.3.2 Wiring `trigger_rst` to the Global Reset

In every protected variant, `trigger_rst` feeds into the design's existing reset tree. `combined_top`'s own local `fault_ctr_trigger_rst` is OR-ed into `rst_IBUF`, the single net from which the entire design's reset tree fans out.

```
1 assign rst_IBUF = rst_IBUF_direct | fault_ctr_trigger_rst;
```

Listing 3.3: Wiring the fault-rate trigger into the global reset.

with the original top-level reset input buffer retargeted to drive `rst_IBUF_direct` instead of `rst_IBUF` directly. An excessive TMR fault rate, as tracked by the sliding-window counter, now triggers a reset of the entire signing operation.

This mechanism is identical across all four protected variants, including output-path-only protection, where `combined_top`'s own flip-flops are left untriplicated but the module still hosts the aggregation, the counter instance, and this reset wiring.

3.4 Protection Strategies

3.4.1 Selection Rationale: Exposure Time

Rather than selecting modules by size or by how “important” they intuitively seem, all three selective strategies are guided by a single concrete criterion: how long a corrupted value would persist before being overwritten again. Computational and pipeline registers (NTT butterfly stages, the polynomial-sampling registers, shift-register-based serialization) are rewritten every clock cycle as part of normal operation. A single-event upset there has a vulnerability window of about one clock period and is naturally overwritten, self-correcting on the very next cycle, regardless of whether it is protected. Final output registers and control-flow state, by contrast, are written once and then held for the remainder of the signing operation, potentially tens of thousands of cycles, while a signature is computed and transmitted. A fault persists for that entire window and directly corrupts the result or the computation's control flow. Protecting the second category is therefore where each protected flip-flop buys the most exposure reduction, which is what all three selective strategies below target.

This rationale has one explicit limitation: a fault in an unprotected intermediate register can still propagate forward into a protected final register. All three of that register's TMR copies would then correctly agree on and store an already corrupted value. Selective protection reduces the modules from which a *permanent, detected* outcome can originate, but it does not eliminate the possibility that an upstream fault contaminates output that looks internally consistent.

3.4.2 Full Protection

Every one of the design's 29,098 flip-flops is replaced, covering every register category that the exposure-time rationale above would otherwise exclude. This is the only variant where no *flip-flop* is left outside TMR protection. Chapter 5 discusses a confirmed exception built from non-flip-flop storage.

3.4.3 Output Path Only

Protects the modules that directly produce the final signature bytes or control the hashing engine on which they depend: `encoder` (final byte packing), `makehint` (hint polynomial, a direct signature component), `gen_c` (challenge polynomial, binds the signature to the message), and all three instantiated variants of the Keccak control FSMs (`keccak_fsm1`, `keccak_fsm2`, `sha3_fsm3`, three copies each since the design instantiates three independent Keccak cores for key expansion, matrix generation, and challenge hashing). This covers approximately 2,385 flip-flops, 8.2% of the total.

3.4.4 Control State Only

Protects only `combined_top`'s own control-state flip-flops (1,351 flip-flops, 4.6% of the total), the top-level finite-state machine that governs every stage of the signing algorithm and, as Section 2.1 identifies, is exactly the kind of logic a loop-abort fault attack relies on. Unlike the strategy of Section 3.4.5 below, this one adds no output-path protection at all: the `encoder`, `makehint`, `gen_c`, and Keccak control FSM modules remain unmodified. It isolates the value of protecting the control-flow FSM in its own right, independent of the output-path cost, and is the cheapest of the four strategies in both flip-flop count and area overhead.

3.4.5 Output Path Plus Control State

Extends the output-path set of Section 3.4.3 with the design's top-level finite-state machine (`combined_top`, 1,351 flip-flops) and `usehint` (2,854 flip-flops), which applies hints to produce final output values with the same long exposure window as the output-path registers described above. This covers approximately 6,590 flip-flops, 22.6% of the total.

Figure 3.2 summarizes all four strategies against `combined_top`'s module hierarchy at a glance, with each color showing which of the four strategies cover a given module.

Several of these modules are reached by no selective strategy at all, and some of what they contain is beyond every strategy, full protection included. Chapter 5 sets out which.

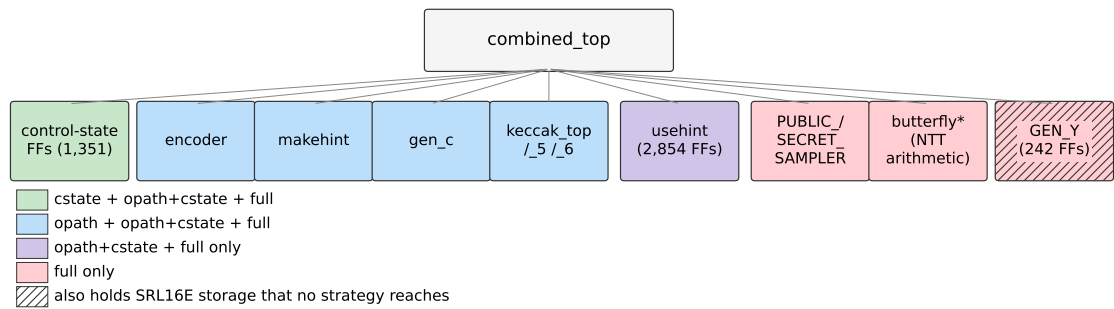


Fig. 3.2.: Protection coverage across combined_top's module hierarchy, by strategy.

3.5 Automated Netlist Transformation

3.5.1 Starting Point: the Post-Synthesis Netlist

Every design variant starts from the same artifact, the post-synthesis gate-level netlist of the unmodified reference design, obtained from Vivado via

```
1 open_run synth_1
2 write_verilog -force -file base_design.v
```

Listing 3.4: Exporting the synthesized netlist.

base_design.v is therefore not RTL. Every flip-flop that synthesis maps onto an FDRE or FDSE primitive already appears as an explicit instance of that primitive, not as inferred behavioral code, which is precisely what makes a purely textual, regular-expression-based transformation of the netlist both possible and reliable.

3.5.2 generate_full_security.py: Full-Design Transformation

generate_full_security.py produces full_security_design.v from base_design.v in two passes. First, every primitive name is replaced unconditionally:

```
1 content = re.sub(r'\bFDRE\b', 'trm_fdre', content)
2 content = re.sub(r'\bFDSE\b', 'trm_fdse', content)
```

Listing 3.5: Primitive replacement (generate_full_security.py).

Second, the script parses the netlist's real module and instantiation hierarchy directly from the text and performs three further edits:

1. Every `trm_*` instance is given a uniquely numbered `fault_wire_N`, connected to it via an added `.fault(fault_wire_N)` port connection, located by finding the line that closes the instance's port list `()`; and inserting the new connection immediately before it.
2. Every module that owns at least one fault wire, or that sits between such a module and `combined_top` in the hierarchy, is given a new `module_fault` output port, OR-reducing its own fault wires together with every child's `module_fault` input. The parent's instantiation of that module gains a matching port connection, threading the aggregate one level upward.
3. At `combined_top`, the design root, this leaves one final OR-reduction of every source that reaches it. That signal drives the design's single `fault_counter` instance, clocked from whichever clock net `combined_top`'s own flip-flops used, falling back to the top-level `clk_IBUF_BUFG` net if none is found.

Applied to the full design, this produces 29,098 fault wires across 138 modules, each containing at least one. Every one of those signals threads up to the single `fault_counter` instance at `combined_top`.

The line-by-line pass above is not the script's last step. A separate post-processing function runs once on the fully assembled output and performs the reset-tree wiring, applying an exact whole-file string replacement to the top-level reset input buffer instantiation. Before replacing it, the function counts how many times the expected pattern occurs and raises an error if that count is not exactly one. The same exact-match-or-fail pattern is used throughout every transformation script.

3.5.3 Selective Transformation: a Module Allow-List

`generate_output_path_design.py` and `generate_output_path_control_state_design.py` call the same function as `generate_full_security.py` directly, passing an explicit `TARGET_MODULES` set in place of the "every module" default.

```
1 TARGET_MODULES = {  
2     'encoder', 'makehint', 'gen_c',  
3     'keccak_fsm1', 'keccak_fsm1_71', 'keccak_fsm1_93',  
4     'keccak_fsm2', 'keccak_fsm2_72', 'keccak_fsm2_94',
```

```

5     'sha3_fsm3',    'sha3_fsm3_73',    'sha3_fsm3_95',
6 }
7 ...
8 if m_decl:
9     in_target_module = m_decl.group(1) in TARGET_MODULES
10 ...
11 if in_target_module:
12     line = re.sub(r'\bFDRE\b', 'trm_fdre', line)
13     line = re.sub(r'\bFDSE\b', 'trm_fdse', line)

```

Listing 3.6: Module allow-list gating: generate_output_path_design.py's TARGET_MODULES, applied by the shared function from Section 3.5.2.

Modules outside this set pass through completely unchanged. Their flip-flops remain standard FDRE/FDSE primitives, with no voter, no fault wire, and no module_fault contribution. This allow-list is the entire mechanism behind two of the four protection strategies, which differ only in which module names populate TARGET_MODULES: the output-path-plus-control-state set additionally includes combined_top and usehint.

generate_control_state_design.py targets only combined_top and does not call the shared function at all. With a single target module that is also the design root there is nothing to thread, since combined_top already owns every fault wire that exists, so the hierarchy-discovery process the other three scripts need would do no useful work here. It instead injects the counter directly, producing the same single-instance-at-combined_top outcome by a simpler path.

Chapter A reproduces generate_full_security.py and the shared library it calls into in full.

Verification and Evaluation

4.1 Functional Verification

4.1.1 Known-Answer-Test Methodology

Before any design variant's power, performance, or area is characterized, it must first be shown to compute the correct signature. Every variant is verified against five Known-Answer-Test vectors at Dilithium security level 2, using the same behavioral testbench throughout. The testbench loads each vector's message and secret key through the design's input handshake, runs the signing operation to completion, and compares the resulting signature word by word against the vector's expected output, reporting any mismatch. A design variant only proceeds to synthesis and implementation once it has passed all five vectors.

4.1.2 Verification Results

Every design variant passes all five vectors, confirming that the netlist-transformation mechanism preserves functional correctness across every protection strategy, from the unprotected baseline to full TMR.

4.1.3 A Verification Finding: the Fault Counter X-Propagation Bug

An unknown value reaching the counter from an unreset security-level-3/5 flip-flop propagated through the counter's arithmetic indefinitely and eventually poisoned the reset tree itself.

That is the reason `fault_s` is needed. Case equality (`===`) is a simulation-only construct: `X` exists only as a simulation abstraction, and Vivado's synthesis reduces `===` to an ordinary `==` accordingly. `fault_s` therefore fixes the gate-level simulation model, not the implemented hardware, which never encounters an `X` to begin with. Substituting it for the raw `fault` input inside the counter's clocked logic makes an

unknown value read as “no fault” in simulation, exactly where this bug surfaced, rather than propagating indefinitely. Because `fault_counter.v` is referenced rather than copied into each generated netlist, the fix applies to every variant at once.

4.1.4 Unit-Level Fault Injection

Beyond the system-level KAT vectors, each TMR flip-flop wrapper has its own standalone testbench (`fdre_tb.v`, `fdse_tb.v`) that instantiates it alongside a `fault_counter` and injects a fault directly. Verilog’s `force/release` keeps one internal copy at a fixed value (first stuck-at-1 and then stuck-at-0) while the other two continue operating normally, simulating a physical upset on a single redundant copy.

The counter’s own threshold and window logic is exercised separately, in isolation from any flip-flop wrapper, by `fault_counter_tb.v`. It drives synthetic fault patterns directly onto the counter’s `fault` input: idle cycles with no fault, isolated single-cycle faults well below `THR_100`, a sustained fault held exactly at the threshold, the window aging out old faults after the fault stops, and a reset applied mid-trigger. All five cases behave as expected, with no false trigger below threshold, a trigger exactly at threshold, and correct clearing both by window aging and by explicit reset.

4.2 Area, Performance, and Power Characterization

The three subsections below report each of the PPA axes across all five protection-scheme variants on Artix-7, taking area first because it is the axis TMR affects most directly and the one the other two partly follow from. A fourth subsection then translates those three figures into throughput and energy per signature, the actual deployable numbers.

4.2.1 Area

Figure 4.1 plots the same numbers as Table 4.2. The near-flat LUT overhead for the three selective strategies against full protection’s much steeper climb is directly visible, and the FF panel’s y-axis, which is over twice as tall as the LUT panel’s, is itself part of the point: TMR’s FF cost simply operates on a different scale from its

Tab. 4.1.: Area, Artix-7 xc7a200tsbv484-1 (133,800 LUTs / 267,600 FFs available).

Resource	base	cstate	opath	opath+cstate	full
Slice LUTs	58,505	61,176	62,445	69,639	109,940
LUT as Logic	53,954	56,625	57,894	65,088	105,389
LUT as Memory	4,551	4,551	4,551	4,551	4,551
Slice Registers (FF)	29,098	31,965	34,033	42,443	87,460
BRAM Tile	26	26	26	26	26
DSP	16	16	16	16	16
Bonded IOB	140	140	140	140	140

Tab. 4.2.: Area overhead relative to base_design.v.

Resource	cstate	opath	opath+cstate	full
Slice LUTs	+4.6%	+6.7%	+19.0%	+87.9%
LUT as Logic	+5.0%	+7.3%	+20.6%	+95.3%
Slice Registers (FF)	+9.9%	+17.0%	+45.9%	+200.6%

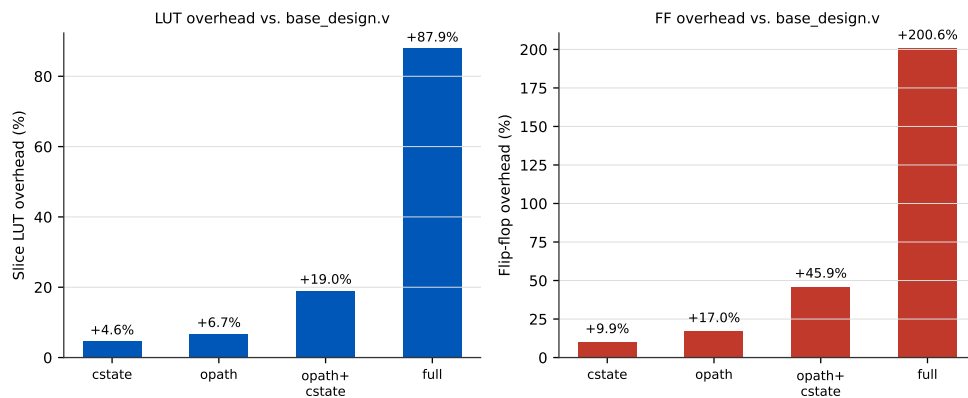


Fig. 4.1.: Area overhead relative to base_design.v (Table 4.2), Artix-7.

LUT cost. LUT as Memory, Bonded IOB, BRAM Tile, and DSP counts are identical across every variant, since TMR triplicates flip-flops and adds voter logic, so only the register count and the LUT count that implements the voters change. The 200.6% FF increase in `full_security_design.v` follows directly from the TMR structure itself: every flip-flop becomes three. The three selective strategies show proportionally smaller overhead that tracks their protected-FF fraction, for reasons the following decomposition makes exact.

Every flip-flop count decomposes exactly, with at most a single flip-flop of residual. Each protected flip-flop becomes three, and the design's one shared `fault_counter` contributes a further 165 flip-flops of its own: a 32-bit count, a 32-bit `running_total`, the 100-cycle window, and `trigger_rst`. This accounts for `control_state_design.v` exactly: 27,747 unprotected flip-flops, plus $3 \times 1,351$ protected, plus the counter's 165, gives precisely the 31,965 observed. The same decomposition reproduces `output_path_design.v` and `output_path_control_state_design.v` to the digit (34,033 and 42,443), and `full_security_design.v` to within a single flip-flop.

Because exactly one `fault_counter` instance exists regardless of protection strategy, its 165-flip-flop cost is fixed. In `full_security_design.v` it contributes just 0.19% of that variant's total flip-flops and 0.28% of its overhead over the baseline. Fault observability is essentially free.

`report_utilization`'s Primitives breakdown confirms the flip-flop count already reported above and surfaces categories that the Slice Logic summary does not. Notably, `base_design.v` instantiates 1,438 SRL16E and 585 SRLC32E shift-register-LUT primitives alongside its 29,098 FDRE/FDSE flip-flops. This study targets only flip-flops, so no strategy reaches any of them.

Tab. 4.3.: Selected primitives, `base_design.v` (`report_utilization`).

Primitive	Count	Category
FDRE	29,087	Flop & Latch
FDSE	11	Flop & Latch
SRL16E	1,438	Distributed Memory (shift register)
SRLC32E	585	Distributed Memory (shift register)
RAMD64E	3,072	Distributed Memory (RAM)
RAMB36E1	26	Block Memory
DSP48E1	16	Block Arithmetic

Tab. 4.4.: Timing, Artix-7 xc7a200tsbv484-1, speed grade -1.

Metric	base	cstate	opath	opath+cstate	full
WNS (ns)	−4.862	−3.751	−5.578	−6.046	−11.681
Failing endpoints	18.7%	21.8%	40.7%	46.0%	64.4%
WHS (ns)	+0.026	+0.029	+0.029	+0.044	+0.009
Max frequency (MHz)	67.3	72.7	64.2	62.3	46.1

4.2.2 Performance

On the Artix-7, the five design variants’ maximum achievable frequency ranges from 46.1 MHz to 72.7 MHz. Figures in this range are expected, since a low-end Artix-7 part was never the reference design’s target. That design is built for high performance and reports its primary results on Virtex UltraScale+, giving only selected Artix-7 and Kintex-7 figures for comparison with prior work [BNG21].

Three of the four TMR variants degrade WNS relative to the baseline by 0.7 ns to 6.8 ns, consistent with the majority voter adding delay to every protected register-to-register path. `control_state_design.v` is the exception. Its WNS, −3.751 ns, is *better* than the unprotected baseline’s −4.862 ns, despite having strictly more logic. The cause is place-and-route solution variance, not a timing benefit from protection. The same effect appears elsewhere in this project, where `output_path_design.v`’s WNS improved after a netlist regeneration that only added combinational logic. The WNS deltas in this table therefore show TMR overhead broadly rather than a precise, monotonic per-flip-flop cost function.

`full_security_design.v`’s own degradation is disproportionately large, −11.681 ns against −5.578 ns and −6.046 ns for the two next-worst variants. The gap holds under a second, independent place-and-route directive, so it is not a single unlucky solution. The most plausible cause is that consolidating every protected module’s fault signal through one long-fanning aggregation tree up to the single shared `fault_counter` creates a harder placement and routing problem.

Hold is met with positive margin in every variant, though `full_security_design.v`’s +0.009 ns margin leaves it the closest of the five to failing. The failing-endpoint percentage rises monotonically with protected-FF fraction across all five variants, reflecting the fraction of paths that include at least one voter, and does not share `control_state_design.v`’s WNS anomaly.

Tab. 4.5.: Total on-chip power by device (W): vectorless estimate, SAIF-informed, and their ratio.

Design	Artix-7			Kintex-7		
	Vectorless	SAIF	Ratio	Vectorless	SAIF	Ratio
base	1.746	0.334	5.23×	1.654	0.268	6.17×
cstate	1.714	0.326	5.26×	1.684	0.276	6.10×
opath	1.610	0.332	4.85×	1.428	0.283	5.05×
opath+cstate	1.717	0.352	4.88×	1.568	0.295	5.32×
full	3.279	0.726	4.52×	n/a	n/a	n/a

4.2.3 Power

Methodology. Every design variant was first characterized with Vivado’s vectorless estimate, deriving switching activity from the clock constraint alone rather than from simulated behavior. A behavioral simulation of the gate-level netlist, covering five KAT vectors per variant, was then captured to a Switching Activity Interchange Format (SAIF) file and fed back into `report_power` (`read_saif -strip_path tb_sign_trmff_top/DUT`), upgrading the confidence level from Low to High. Between 43% and 61% of each design’s nets were matched by the SAIF capture, with the remainder still falling back to the vectorless default internally, so even the High-confidence figures above are a hybrid rather than a fully activity-informed measurement. The SAIF-informed report reflects the design’s actual switching activity for whichever nets the capture matched, while the vectorless estimate is always a statistical approximation based on an assumed toggle rate.

Tab. 4.6.: Static versus dynamic power, Artix-7: vectorless estimate versus SAIF-informed. Vivado rounds each component independently, so a column need not sum exactly to the total in Table 4.5; `base`’s SAIF-informed pair differs from its 0.334 W total by 0.001 W for this reason.

	base	cstate	opath	opath+cstate	full
Dynamic, vectorless (W)	1.607	1.575	1.471	1.578	3.133
Static, vectorless (W)	0.139	0.139	0.138	0.139	0.146
Dynamic, SAIF-informed (W)	0.202	0.202	0.209	0.228	0.601
Static, SAIF-informed (W)	0.133	0.124	0.124	0.124	0.125

Static power is essentially insensitive to the protection level, so the overhead is almost entirely a dynamic cost. It stays within 0.124–0.133 W SAIF-informed across all five designs despite `full_security_design.v` being nearly twice the size of the smallest (109,940 vs. 58,505 LUTs), consistent with leakage on a fixed die being governed by the device rather than by how much of it a design occupies. Dynamic power instead scales with protection level, 0.202 W to 0.601 W SAIF-informed, a 3.0× increase from baseline to full.

The vectorless estimate exceeds the SAIF-informed estimate by roughly $4.5\times$ to $5.3\times$, consistent with its flat $\sim 12.5\%$ toggle-rate assumption against this design's FSM-driven, bursty activity, where most of the datapath idles until a specific operation activates it. The factor shrinks as coverage grows, from $5.2\times$ – $5.3\times$ at 0–4.6% TMR to $4.5\times$ – $4.9\times$ at 8.2–100%, plausibly because triplication leaves a larger fraction of the design genuinely exercised.

Rankings among closely-spaced variants are not stable across estimation methods. Vectorless ranks the four non-full variants `opath` (1.610 W) < `cstate` (1.714 W) < `opath+cstate` (1.717 W) < `base` (1.746 W), while SAIF-informed ranks them `cstate` (0.326 W) < `opath` (0.332 W) < `base` (0.334 W) < `opath+cstate` (0.352 W). None of the four holds its position between methods, reordering across a spread of roughly 8% under either one. `full_security_design.v` alone is unambiguously highest under both, its 109,940 LUTs and 87,460 flip-flops putting it far outside the 58,000–71,000 LUT range of the rest.

A hierarchical breakdown shows where the estimation method changes the picture. Vivado attributes only *dynamic* power to hierarchy, so `combined_top` accounts for the whole 1.607 W dynamic component of `base_design.v`'s vectorless total. Within it the ranking inverts between methods. The three KECCAK instances dominate the vectorless estimate at 0.992 W (62%) against the seven BRAM instances, which hold the 26 RAMB36E1 tiles, at 0.168 W (10%). Under SAIF the BRAM instances lead with 0.107 W (53%) while KECCAK falls to 0.036 W (18%). Keccak idles outside active hash operations, while BRAM access sustains switching closer to what the vectorless default assumes. The reordering was confirmed only for `base_design.v`.

4.2.4 Throughput and Energy per Signature

Watts alone do not describe what a signing accelerator costs to deploy. The deployable figures are signatures per second and energy per signature, which is also how the reference design reports itself [BNG21], and both follow from data already presented. Round 3 signing is deterministic, so a given key and message always take the same number of cycles. Those counts are identical across all five variants and both devices at 10,945, 22,581, 22,584, 16,789 and 22,609 for the five KAT vectors, a mean of 19,102. The spread between vectors comes from rejection sampling, not from protection. TMR adds no cycles at all, as follows from the vote being combinational, and this was measured on every variant rather than assumed.

Energy per signature has two parts that respond differently to clock frequency. Switching energy is set by how many transitions a signature causes, so it does

Tab. 4.7.: Throughput and energy per signature.

Design	$f_{\max}$ (MHz)	Time/sig (μ s)	Signatures/s	Energy/sig (μ J)
base	67.3	283.8	3,523	76.3
cstate	72.7	262.7	3,806	71.2
opath	64.2	297.5	3,361	76.8
opath+cstate	62.3	306.6	3,261	81.6
full	46.1	414.4	2,413	166.6

not fall when the clock slows, whereas leakage accrues over whatever wall-clock time the operation occupies. Equation (2.4) implies the first half, and re-running `report_power` on one fixed routed design while varying only the clock constraint from 8 to 20 ns confirms it for the tool’s own model, leaving dynamic energy per cycle constant at 16.07 nJ to within 0.09%. Table 4.7 therefore charges switching energy at the 100 MHz point where it was measured and leakage over each variant’s own $1/f_{\max}$ runtime.

Full protection is the only row separated from noise, costing $2.18\times$ the baseline’s energy per signature and giving up 31.5% of its throughput. Since TMR consumes no extra cycles, that throughput loss is entirely voter delay eating $f_{\max}$.

4.3 Cross-Device Comparison

Motivated by the observation that a single-device characterization cannot distinguish a TMR-specific cost from a device-specific one, four of the five design variants were additionally characterized on a Kintex-7 part, xc7k160tiffv676-2L. Kintex-7 is a natural second family here because the reference design reports Kintex-7 results of its own [BNG21], which keeps the comparison close to the hardware this study modifies.

`full_security_design.v` needs 109,940 LUTs, which exceeds this part’s 101,400 LUT capacity, so it could not be characterized here at all. This is a capacity limit, not a timing or power result. The Kintex-7 part actually used runs a faster speed grade (-2L) than the Artix-7 part’s -1, so the comparison below conflates a device-family effect with a speed-grade effect.

Every design that fits achieves a substantially higher maximum frequency on Kintex-7 than on Artix-7: 110.4 MHz versus 67.3 MHz for the baseline, narrowing to 100.9 MHz versus 62.3 MHz for output-path-plus-control-state. Every design also draws slightly less estimated power on Kintex-7 under both methods.

Tab. 4.8.: WNS and maximum achievable frequency by device.

Design	Artix-7 WNS	Kintex-7 WNS	Artix-7 $f_{\max}$	Kintex-7 $f_{\max}$
base	−4.862 ns	+0.945 ns	67.3 MHz	110.4 MHz
cstate	−3.751 ns	+0.274 ns	72.7 MHz	102.8 MHz
opath	−5.578 ns	+1.006 ns	64.2 MHz	111.2 MHz
opath+cstate	−6.046 ns	+0.086 ns	62.3 MHz	100.9 MHz
full	−11.681 ns	n/a	46.1 MHz	n/a

More notably, the vectorless-to-SAIF ratio splits into two clusters on *both* families at the same boundary, namely whether the output-path modules are protected. base and cstate run $5.2\times$ – $5.3\times$ on Artix-7 and $6.1\times$ – $6.2\times$ on Kintex-7, opath and opath+cstate $4.85\times$ – $4.88\times$ and $5.05\times$ – $5.32\times$. This is the only pattern in the nine-combination dataset that reproduces identically on both families. Its mechanism is unconfirmed, though these modules’ serialization shift registers toggle far more regularly than the control-flow-driven logic elsewhere, which the vectorless default may model differently.

The Kintex-7 frequency trend is not monotonic. Maximum frequency runs 110.4, 102.8, 111.2 and 100.9 MHz at 0%, 4.6%, 8.2% and 22.6% protected:

output_path_design.v sits above the unprotected baseline despite having strictly more logic, the same place-and-route solution variance already documented for control_state_design.v on Artix-7, now visible on this device family too. Only the two endpoints land in the order more protection would predict, least-protected highest and most-protected lowest, while the two middle variants reorder. What survives is the qualitative floor rather than a specific shape: every variant that fits stays comfortably above 100 MHz, well clear of any pass/fail threshold.

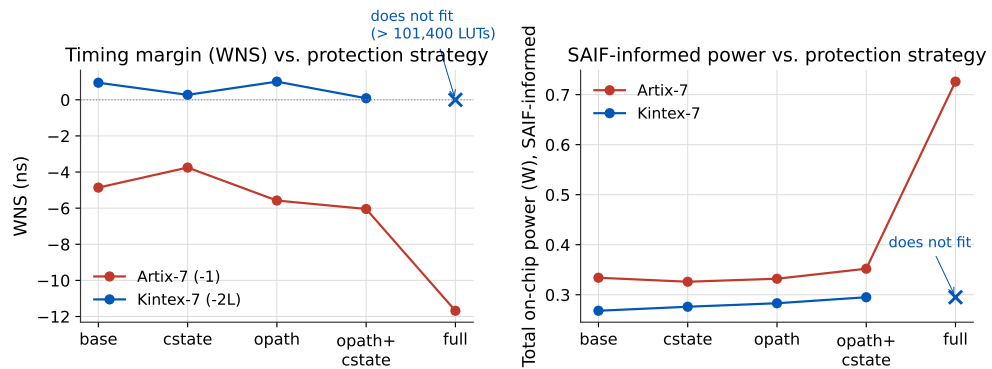


Fig. 4.2.: WNS and SAIF-informed power against protection strategy.

Figure 4.2 shows both trends side by side. The left panel’s Kintex-7 line dips from base to cstate, spikes back above base at opath, then dips again at opath+cstate,

the same reordering of the two middle variants described above. In the right panel, Kintex-7's SAIF-informed power rises monotonically with protection level, while Artix-7's dips slightly at `cstate` rather than at `opath`, then climbs through `opath` and `opath+cstate` before full protection's sharp jump.

4.4 Fault-Rate Observability

This study's evidence that the fault-detection mechanism actually works comes at two distinct levels.

Unit level, confirmed. Section 4.1.4 already establishes both halves of this directly: the majority voter tracking the correct value throughout an injected single-copy fault while the `fault` signal flags the disagreement, and the counter's own threshold and window logic, tested in isolation with synthetic patterns, never false-triggering below `THR_100`, triggering exactly at it, and clearing correctly by both window aging and reset.

System-level fault injection, confirmed. Every functional-verification run exercises the fault counter under correct, unfaulted operation. Across five design variants and five KAT vectors each, `trigger_rst` never asserted and no spurious reset occurred, confirming that the counter does not false-trigger during normal operation, a necessary precondition for the counter to be useful at all, but not the same claim as fault detection. A dedicated testbench, `rtl_tb/tb_sign_trmff_fault_injection_top.v`, closes that gap. It forces a stuck-at fault directly onto a real register during an actual signing operation, then checks whether the resulting signature still matches its KAT vector. Six cases span four design variants and three registers, summarized in Table 4.9.

Tab. 4.9.: System-level fault-injection results.

Case	Target	Variant	Category	Result
1	GEN_Y SEED_SIPO_reg	full (100%)	Vulnerability	CORRUPTED
2	ctrfsml	cstate (4.6%)	Masking	MASKED
3	ctrfsml	base (0%)	Vulnerability	CORRUPTED
4	encoder PISO_reg	opath (8.2%)	Masking	MASKED
5	encoder PISO_reg	cstate (4.6%)	Vulnerability	CORRUPTED
6	encoder PISO_reg	opath (8.2%)	Common-mode	CORRUPTED

Three vulnerability cases (1, 3, 5), all on unprotected plain registers, reached full confirmation. Since a plain register carries no fault signal to trip `fault_counter`, the injected fault could be held for the register's entire active window without risk,

ruling out a missed-timing or lucky-coincidence explanation. Case 5's unprotected encoder output register produced 303 of 303 output words wrong, an unambiguous, complete corruption. Case 3 is stronger still. Sustained corruption of the unprotected signing-attempt counter (`ctr fsm1`) did not merely produce a wrong signature; it caused a permanent deadlock, confirmed via a heartbeat monitor showing `ctr fsm1` frozen and `valid_o` never asserting across four readings spanning 1.5 ms of simulated time, an availability failure and not only a correctness one, from a single unprotected register.

Case 1 is this campaign's headline result. Forcing a fault onto `GEN_Y`'s `SEED_SIP0_reg`, the SRL16E-based register that lies structurally outside every variant's TMR coverage, corrupts 302 of 303 output words *even in the full-protection variant*, the one design point where every ordinary flip-flop is triplicated. This is a directly observed, reproducible demonstration that the TMR flip-flop-replacement approach has a real blind spot for SRL-based storage that persists at full coverage.

Cases 2 and 4, the two masking cases on TMR-protected registers, both produced a correct signature. Each required establishing that the injection genuinely overlapped the register's active window, and the two cases differ in how well that overlap is actually established. Case 4 used the offset that case 6's calibration independently proves overlaps `PIS0_reg[0]`'s real read cycle, confirmed by direct instrumentation, so a correct signature there cannot be explained by a timing miss. Case 2's read cycle is not recoverable from the netlist alone: tracing `ctr fsm1`'s connections to `GEN_Y` yields only an output, which carries no read-timing information. It instead held the fault across `ctr fsm1`'s entire active window, gated by the same `state==UNLOAD_Z` testbench signal that the window-placement check below shows lags `PIS0_reg`'s actual read timing by roughly 83,630 ns. Whether the same kind of lag applies to `ctr fsm1` was never independently instrumented, so case 2's MASKED result rests on an unverified windowing assumption that case 4's does not share.

For that one diagnostic run, the fault counter's reset output was decoupled from the reset network so the guaranteed `THR_100` crossing could not desync the testbench; `fault_counter`'s own threshold logic was left untouched and verified to still assert and clear correctly. Cases 1, 3, 4, 5, and 6 rest on directly verified timing. Case 2 alone carries this residual assumption.

A further, separate test confirms the fault counter's reset mechanism end-to-end. A sustained fault held long enough to cross `THR_100` on a protected `ctr fsm1` register causes `fault_ctr_trigger_rst` and `rst_IBUF` to assert together at the tenth faulty cycle, and both clear together once the sliding window ages the fault back out after the injection is released. This is the complete assert-then-clear lifecycle observed

directly inside a real signing operation, previously demonstrated only in the isolated `fault_counter_tb.v` unit test.

Case 6 extends the campaign from single-copy faults to a *common-mode* fault, in which one physical event corrupts more than one redundant copy at once. `vote3`'s plain majority function has no way to distinguish that from an ordinary single-copy disagreement: if two of three copies are corrupted to the same wrong value, the vote outputs that wrong value regardless of which copy is actually correct. Reusing case 4's setup, both `q_a` and `q_b` of the protected encoder `PISO_reg[0]` were forced to the same value simultaneously, producing `CORRUPTED` at the identical bit position as an unprotected calibration run, which rules out coincidence. Forcing two of three redundant copies to the same wrong value therefore defeats TMR exactly as `vote3`'s majority logic predicts, a direct and reproducible confirmation of the common-mode limitation. This confirms the mechanism on one register, one bit, and one corruption pattern; a different register, a multi-bit fault, or two copies corrupted to two different wrong values were not tried.

Window placement is critical for this register, and the same constraint applies to any injection targeting the output path. The testbench's sequencer signals readiness at $t = 20,780$ ns, but instrumenting the transfer handler to print each output transfer's timestamp shows the first real transfer at $t = 104,410$ ns, an $\sim 83,630$ ns gap that the earlier per-word timing estimate had averaged away. Injections placed at the earlier point return `MASKED`, while a calibration run on the equivalent unprotected register at the corrected offset produces a clean `CORRUPTED`, which isolates window placement as the cause.

Discussion

5.1 Research Question Revisited

Section 1.2 posed one question with two parts: what power, performance, and area impact TMR has on this hardware, applied both fully and selectively. The measured results answer each part in turn.

The cost of full protection. Full TMR costs +87.9% Slice LUTs, +200.6% flip-flops, a WNS degradation of 6.819 ns, and, under the SAIF-informed power estimate, a 117% increase in total power, all on Artix-7. It is the most expensive strategy evaluated by a wide margin on every metric measured. On Kintex-7 it is not merely expensive; it does not fit on the available device at all.

Another version of this design gave every protected module its own local `fault_counter` instead of one shared instance. Figure 5.1 shows the structural difference. Table 5.1 compares full protection's measured figures under both architectures.

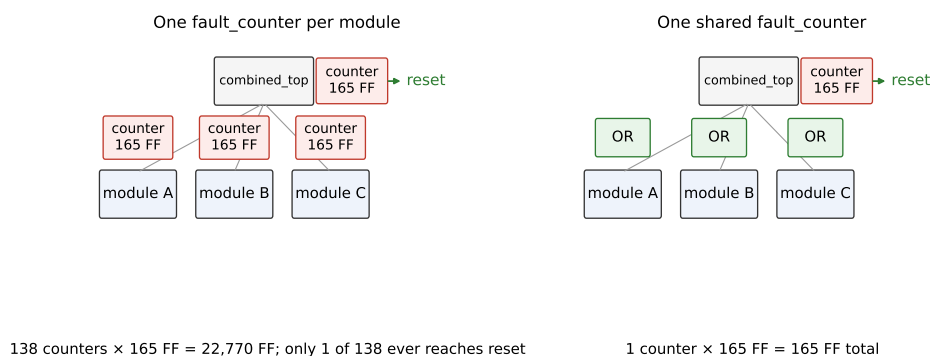


Fig. 5.1.: Fault-counter placement: one instance per protected module versus one shared instance at `combined_top`.

Area improves under the shared counter, since a single 165-flip-flop counter replaces 138 of them. Timing and power both get worse: WNS degradation rises from 2.655 ns to 6.819 ns and the power increase rises from 87% to 117%. The two architectures also differ in what a threshold crossing means. Per-module counters require one module's own fault rate to cross `THR_100` on its own, whereas the shared

Tab. 5.1.: Full protection: one `fault_counter` per module versus one shared instance.

Metric	Per-module counter	Shared counter
Slice LUT overhead	+90.8%	+87.9%
Flip-flop overhead	+278.3%	+200.6%
WNS degradation	2.655 ns	6.819 ns
SAIF-informed power increase	87%	117%

counter aggregates across the whole design, so a disturbance spread thinly over many modules triggers a reset under the shared design but can go unreported under the per-module one. The redesign is a trade, not a uniform improvement.

Selective protection's trade-off. Selective protection achieves meaningfully lower overhead while remaining functionally correct, and the evidence for it arrives in layers. All three selective strategies pass every KAT vector at overhead proportional to their protected-flip-flop fraction, from `control_state_design.v`'s +9.9% flip-flops up to `output_path_control_state_design.v`'s +45.9%. That establishes functional correctness, not fault resistance. Unit-level injection then confirms the masking guarantee for an individual flip-flop in isolation, and the system-level campaign confirms it inside real signing operations, where unprotected registers reliably corrupt the signature or, in one case, hang the operation outright. The caveat is that none of this measures how much attack-relevant state a given strategy leaves exposed, which Section 5.2 sets out.

Cross-device consistency. A single-device characterization cannot rule out a device-specific artifact. Section 4.3 shows the same underlying mechanism, voter logic degrading timing margin, operating on both Artix-7 and Kintex-7, though not with an identical shape, and not the shape it had before the single-counter redesign either. Table 5.2 gives both. Before the redesign, Kintex-7 WNS fell monotonically with protected-flip-flop fraction, a large initial drop followed by a plateau, the cleanest trend measured anywhere in this study. After it, only the two endpoints land in the order more protection would predict, least-protected best and most-protected worst; the two middle variants reorder, the same place-and-route noise already established for Artix-7's own closely-spaced variants. What generalizes is the mechanism. The specific numbers, the exact shape of the trend, and whether a given design meets a given frequency target do not. The redesign itself is a reminder that even the shape of a trend can be an artifact of one specific netlist.

Tab. 5.2.: Kintex-7 WNS (ns), per-module counter versus shared counter. base has no counter and cstate has exactly one under either architecture, so only the two middle variants change.

Design	Per-module counter	Shared counter
base	+0.945	+0.945
cstate	+0.274	+0.274
opath	+0.075	+1.006
opath+cstate	+0.071	+0.086

5.2 Limitations

System-level confirmation is scoped. This campaign covers four of the five variants, three registers, one KAT vector at security level 2, one fault model, and a single common-mode instance. It demonstrates that TMR’s masking guarantee holds for those specific cases in a real signing operation, not that it holds for every flip-flop in every protected module under every possible input.

Coverage stops at flip-flops. A flip-flop transformation reaches only flip-flops, and several mechanisms resist TMR protection outright regardless of strategy. The Fiat-Shamir-with-Aborts bound check’s comparator is fused directly into the NTT butterfly arithmetic by synthesis rather than left as a standalone module, so it is combinational logic with no flip-flop for any strategy, full protection included, to act on. The storage bit it feeds, `norm_rejected_reg`, is a separate, ordinary flip-flop, incidentally protected wherever `combined_top` itself is a target module. The seed-expansion pipeline’s masking-vector storage is a sharper case: it synthesizes as SRL16E primitives rather than FDRE/FDSE cells, placing it beyond every strategy’s reach, full protection included. SRL16E is not an isolated instance; there are 1,438 of them in `base_design.v` alone. Direct fault injection confirms the consequence empirically rather than only as a netlist-level argument: forcing a fault onto this exact register corrupts 302 of 303 output words even in the full-protection variant. Counted by instantiated storage capacity, the 29,098 flip-flops even full protection reaches are about 2% of the design’s total, and discounting block RAM entirely, on the pessimistic assumption that it is largely unoccupied, non-flip-flop storage is still roughly 89% of the remainder. Register-based TMR protects register state, and register state is a minority of the state this design holds.

The coverage side of the trade-off is not quantified. Cost is measured here to three decimal places, but coverage is reported only as a fraction of flip-flops protected and as six fault-injection cases. Those cases establish that masking occurs where claimed and that specific blind spots exist. They do not establish what proportion of

attack-relevant state a given strategy covers. A claim of the form “+9.9% flip-flops buys $X\%$ of the exploitable state” would require a fault model over the whole design and an enumeration of attack-relevant registers, neither of which this study attempts.

No multibit-fault-rate granularity. The fault counter’s OR-reduction treats any number of simultaneously faulting flip-flops within a module as a single event. A lone single-bit upset and a wider disturbance affecting many flip-flops in the same cycle both register as exactly one faulty cycle in the sliding window. The counter reports an elevated fault *rate* over time but not fault *multiplicity* within a single cycle, so it cannot by itself distinguish an isolated, cosmic-ray-style upset from a broader glitch hitting many registers at once.

WNS alone is a noisy signal for closely-spaced Artix-7 variants. Section 4.2.2 already attributes `control_state_design.v`’s better-than-baseline WNS to place-and-route solution variance rather than a real timing benefit; that variance is measured here, not assumed. Re-implementing both variants under three timing-driven `place_design/route_design` directive combinations (Default, Explore, RuntimeOptimized) produced a WNS spread of 1.385 ns for the baseline and 1.718 ns for `control_state_design.v`. Both are wider than the 1.111 ns gap in question, which is therefore not distinguishable from ordinary place-and-route variance. The underlying conclusion was never in doubt; the failing-endpoint percentage and the Kintex-7 cross-device frequency trend independently support the same mechanism. What is in question is only WNS’s reliability as a single-worst-path metric for this one pairwise comparison.

Cross-device comparison conflates two effects. Section 4.3 already notes this: the Kintex-7 part actually characterized runs a faster speed grade (–2L) than the Artix-7 part (–1), so the device-family effect and the speed-grade effect are not separated. A –1-speed-grade Kintex-7 part was not available.

5.3 Future Work

The limitations above point directly to concrete next steps.

Extend system-level fault injection to more registers and KAT vectors. The confirmed cases cover three registers under a single KAT vector at security level 2. Applying the same methodology, direct read-timing confirmation where a register’s active window can be established from the netlist and the reset-decoupling fallback

where it cannot, to additional registers, KAT vectors, and security levels would extend this study's empirical coverage of TMR's masking guarantee beyond the specific cases tested here.

A narrowly scoped, fully complete full-protection variant. Closing the SRL16E/SEED_SIP0_reg coverage gap was considered via a modified synthesis step: raising `synth_design's -shreg_min_size` above the longest shift register in the design forces shift-register RTL into ordinary flip-flops instead of SRL primitives. Applying it globally was rejected, since it changes `base_design.v` itself, the shared ancestor of every protection strategy on both device families, and would require re-characterizing all nine design-device combinations rather than the subset a narrower fix would touch. Given this study's focus on the PPA cost of TMR rather than on maximizing this reference design's coverage, that cost was judged disproportionate, and the fix would in any case be partial, since distributed and block RAM storage would remain uncovered.

A narrower version sidesteps the re-characterization cost: apply the flag only when generating one new, additional `full_security_design.v`-derived netlist, leaving `base_design.v` and the other eight existing characterizations untouched. With 1,438 SRL16E and 585 SRLC32E instances in `base_design.v`, the resulting variant would cover every flip-flop *and* every shift-register LUT, a genuinely complete full-protection point that closes the specific SEED_SIP0_reg gap case 1 demonstrates, at the cost of one additional characterization.

A per-cycle popcount fault counter. An earlier design iteration computed a popcount over the individual, non-OR-reduced fault wires every cycle, supporting a second, finer-grained threshold, `THR_CYCLE`, that could trigger a reset if enough flip-flops faulted in the very same cycle, directly distinguishing a single-bit upset from a multibit event. It was not used for any result for one concrete, measured reason: computing a popcount over potentially hundreds of fault wires per module requires an adder tree scaled to that width in every protected module, which measurably increased LUT usage over the OR-reduced design, an unacceptable cost given that area overhead is part of the stated research question. Revisiting it, perhaps scoped to only the highest-value modules, would close the granularity gap above without necessarily re-imposing its full area cost everywhere.

Extend SAIF coverage to additional security levels. The completed SAIF cross-check captures five KAT vectors per variant, but all under security level 2 only, and matched only 43–61% of each design's nets. This is enough to move the confidence classification from Low to High, but not enough to claim representative coverage of the design's full range of operating conditions.

Test whether more timing margin removes the Artix-7 WNS noise. The measured place-and-route noise floor confirms that the gap between the baseline and `control_state_design.v` is noise, but the Artix-7 designs are also already failing timing by several nanoseconds at the 100 MHz constraint used throughout, a regime where a true but smaller per-strategy timing cost could be swamped by that same variance. Re-implementing the Artix-7 variants against a more achievable target frequency, closer to what the Kintex-7 results show these designs can reach, would test directly whether more margin lets a cleaner trend emerge, though the Kintex-7 results themselves suggest even a relaxed constraint may not fully remove the reordering among closely-spaced variants.

Isolate the Kintex-7 speed-grade effect. Re-running at least one design variant on a -1-speed-grade Kintex-7 part would separate the device-family effect this study observes from the speed-grade effect it currently cannot rule out.

Extend the device comparison to newer FPGA families. Whether a Zynq Ultra-Scale+ MPSoC's programmable-logic fabric shows the same overhead trend as the two 7-Series families characterized here is an open question.

Repeat the characterization on an open toolchain. Every figure in this study comes from Vivado's own estimators. Running the same flow through Yosys and OpenROAD would test how much of the measured trend is a property of the designs themselves rather than of one vendor's synthesis, placement, and power models.

Conclusion

This study implemented and characterized Triple Modular Redundancy on a CRYSTALS-Dilithium signing core, asking what TMR costs in area, timing margin, and power when applied both fully and selectively to post-quantum cryptographic hardware. An automated Python transformation retrofits TMR onto an unmodified, synthesized gate-level netlist, four resulting protection strategies were verified against Known-Answer-Test vectors, and all five design variants were characterized through FPGA synthesis and implementation on two device families.

The result is a clear cost gradient. Full protection is the most expensive strategy on every metric measured (+87.9% Slice LUTs, +200.6% flip-flops, a 6.819 ns WNS degradation, and a 117% SAIF-informed power increase on Artix-7) and does not fit on Kintex-7 at all. Selective protection, targeting the held output registers and control-flow state identified by the exposure-time criterion, scales its overhead with the fraction of flip-flops protected, from control-state-only's +9.9% up to output-path-plus-control-state's +45.9%. The underlying mechanism, voter logic degrading timing margin as the protected-flip-flop fraction grows, is visible on both device families evaluated, which is evidence that it is a property of TMR rather than of one device. Its precise shape is not: on Kintex-7 only the two endpoints fall in the order more protection would predict, while the two middle variants reorder within place-and-route noise.

A system-level fault-injection campaign confirms that mechanism where it is applied and, just as clearly, where it is not. Protected registers mask an injected fault; unprotected ones reliably corrupt the signature or hang the operation outright; a common-mode fault defeats the majority vote even on a protected register; and storage that synthesis maps onto shift-register primitives rather than flip-flops stays corruptible even under full protection. One further limit is structural rather than demonstrated: the majority voter is not itself triplicated and remains a single point of failure, which follows from the construction and was not tested here.

Within that scope, a hardware designer can trade cost against flip-flop coverage deliberately, near-complete coverage at a steep price or a targeted fraction of it at a fraction of the cost, rather than facing an all-or-nothing decision. The cost side of

that trade is quantified precisely here; what any given coverage level actually buys against a real attacker is not.

Bibliography

- [Abd+25] Mohamed Abdelmonem, Lukas Holzbaur, Håvard Raddum, and Alexander Zeh. *Efficient Error Detection Methods for the Number Theoretic Transforms in Lattice-Based Algorithms*. Cryptology ePrint Archive, Paper 2025/170. <https://eprint.iacr.org/2025/170>. 2025 (cit. on p. 12).
- [Bai+21] Shi Bai, Léo Ducas, Eike Kiltz, et al. *CRYSTALS-Dilithium: Algorithm Specifications and Supporting Documentation (Version 3.1)*. Round-3 submission to the NIST Post-Quantum Cryptography Standardization Project. <https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf>. 2021 (cit. on pp. 2, 7, 8).
- [BNG21] Luke Beckwith, Duc Tri Nguyen, and Kris Gaj. “High-Performance Hardware Implementation of CRYSTALS-Dilithium”. In: *2021 International Conference on Field-Programmable Technology (ICFPT)*. <https://eprint.iacr.org/2021/1451>. 2021, pp. 1–10 (cit. on pp. 2, 7, 12, 31, 33, 34).
- [BP18] Leon Groot Bruinderink and Peter Pessl. “Differential Fault Attacks on Deterministic Lattice Signatures”. In: *IACR Transactions on Cryptographic Hardware and Embedded Systems 2018.3* (2018). <https://eprint.iacr.org/2018/355>, pp. 21–43 (cit. on pp. 1, 3, 5, 7).
- [Can+19] Matthew J. Cannon, Andrew M. Keller, Hayden C. Rowberry, et al. “Strategies for Removing Common Mode Failures From TMR Designs Deployed on SRAM FPGAs”. In: *IEEE Transactions on Nuclear Science 66.1* (2019), pp. 207–215 (cit. on p. 12).
- [Cho+25] Siddhartha Chowdhury, Nimish Mishra, Sarani Bhattacharya, and Debdeep Mukhopadhyay. “Unified FPGA Design of Kyber and Dilithium with Provable Fault Tolerance”. In: *2025 IEEE 36th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. <https://eprint.iacr.org/2026/1008>. 2025, pp. 158–165 (cit. on p. 12).
- [Duc+18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, et al. “CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme”. In: *IACR Transactions on Cryptographic Hardware and Embedded Systems 2018.1* (2018), pp. 238–268 (cit. on p. 7).
- [Esp+17] Thomas Espitau, Pierre-Alain Fouque, Benoît Gérard, and Mehdi Tibouchi. “Loop-Abort Faults on Lattice-Based Fiat-Shamir and Hash-and-Sign Signatures”. In: *Selected Areas in Cryptography – SAC 2016*. Vol. 10532. Lecture Notes in Computer Science. <https://eprint.iacr.org/2016/449>. Springer, 2017, pp. 140–158 (cit. on pp. 1, 5, 12).

- [KW21] Andrew M. Keller and Michael J. Wirthlin. “Partial TMR for Improving the Soft Error Reliability of SRAM-Based FPGA Designs”. In: *IEEE Transactions on Nuclear Science* 68.5 (2021), pp. 1023–1031 (cit. on p. 11).
- [LSG22] Georg Land, Pascal Sasdrich, and Tim Güneysu. “A Hard Crystal – Implementing Dilithium on Reconfigurable Hardware”. In: *Smart Card Research and Advanced Applications (CARDIS 2021)*. Vol. 13173. Lecture Notes in Computer Science. <https://eprint.iacr.org/2021/355>. Springer, 2022, pp. 210–230 (cit. on p. 12).
- [Lyu09] Vadim Lyubashevsky. “Fiat-Shamir with Aborts: Applications to Lattice and Factoring-Based Signatures”. In: *Advances in Cryptology – ASIACRYPT 2009*. Vol. 5912. Lecture Notes in Computer Science. Springer, 2009, pp. 598–616 (cit. on p. 7).
- [Nat24] National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard*. Tech. rep. FIPS 204. <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.204.pdf>. NIST, Aug. 2024 (cit. on pp. 1, 2, 7).
- [Neu56] John von Neumann. “Probabilistic Logics and the Synthesis of Reliable Organisms from Unreliable Components”. In: *Automata Studies*. Annals of Mathematics Studies 34. Princeton University Press, 1956, pp. 43–98 (cit. on p. 6).
- [Pra+06] Brian Pratt, Michael Caffrey, Paul Graham, Keith Morgan, and Michael Wirthlin. “Improving FPGA Design Robustness with Partial TMR”. In: *2006 IEEE International Reliability Physics Symposium Proceedings*. 2006, pp. 226–232 (cit. on p. 11).
- [SRK04] Praveen Kumar Samudrala, Jeremy Ramos, and Srinivas Katkoori. “Selective Triple Modular Redundancy (STMR) Based Single-Event Upset (SEU) Tolerant Synthesis for FPGAs”. In: *IEEE Transactions on Nuclear Science* 51.5 (2004), pp. 2957–2969 (cit. on p. 11).
- [Uli+23] Vincent Quentin Ulitzsch, Soundes Marzougui, Alexis Bagia, Mehdi Tibouchi, and Jean-Pierre Seifert. “Loop Aborts Strike Back: Defeating Fault Countermeasures in Lattice Signatures with ILP”. In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2023.4 (2023), pp. 367–392 (cit. on p. 12).

List of Figures

2.1	Setup-slack calculation for one register-to-register path.	10
3.1	Block diagram of <code>trm_fdre</code> (Listing 3.1). C, CE, and R fan out identically to all three and are omitted for clarity.	17
3.2	Protection coverage across <code>combined_top</code> 's module hierarchy, by strategy.	23
4.1	Area overhead relative to <code>base_design.v</code> (Table 4.2), Artix-7.	29
4.2	WNS and SAIF-informed power against protection strategy.	35
5.1	Fault-counter placement: one instance per protected module versus one shared instance at <code>combined_top</code>	39
A.1	Fault counter sliding window: rise to <code>THR_100=8</code> and the subsequent 100-cycle slide-out, traced from the internal <code>running_total</code> register. .	66
A.2	Digital waveform of the <code>\$monitor</code> trace in Listing A.5: Q tracks the majority value throughout the stuck-at-1 (500–600 ns) and stuck-at-0 (600–700 ns) injection, while <code>trigger_rst</code> asserts at 685 ns and clears at 1515 ns.	68

List of Tables

4.1	Area, Artix-7 xc7a200tsbv484-1 (133,800 LUTs / 267,600 FFs available).	29
4.2	Area overhead relative to base_design.v.	29
4.3	Selected primitives, base_design.v (report_utilization).	30
4.4	Timing, Artix-7 xc7a200tsbv484-1, speed grade -1.	31
4.5	Total on-chip power by device (W): vectorless estimate, SAIF-informed, and their ratio.	32
4.6	Static versus dynamic power, Artix-7: vectorless estimate versus SAIF-informed. Vivado rounds each component independently, so a column need not sum exactly to the total in Table 4.5; base's SAIF-informed pair differs from its 0.334 W total by 0.001 W for this reason.	32
4.7	Throughput and energy per signature.	34
4.8	WNS and maximum achievable frequency by device.	35
4.9	System-level fault-injection results.	36
5.1	Full protection: one fault_counter per module versus one shared instance.	40
5.2	Kintex-7 WNS (ns), per-module counter versus shared counter. base has no counter and cstate has exactly one under either architecture, so only the two middle variants change.	41

List of Listings

3.1. TMR wrapper for FDRE.	16
3.2. OR-reduced sliding-window fault counter.	18
3.3. Wiring the fault-rate trigger into the global reset.	20
3.4. Exporting the synthesized netlist.	23
3.5. Primitive replacement (generate_full_security.py).	23
3.6. Module allow-list gating: generate_output_path_design.py's TARGET_MODULES, applied by the shared function from Section 3.5.2.	24
A.1. Post-synthesis netlist transformation for full TMR protection (gener- ate_full_security.py).	55
A.2. Shared hierarchy-discovery and single-counter wiring library (sin- gle_counter_lib.py).	56
A.3. TMR wrapper for FDSE.	64
A.4. Console output, fault_counter_tb.v, all five tests passing.	65
A.5. \$monitor trace excerpt around fault injection, full run is 1000 lines over 5 μ s.	67
A.6. Case selection macros and the common-mode force branch.	68
A.7. Console output, reset-mechanism test: fault_ctr_trigger_rst/rst_IBUF asserting and clearing inside a real signing operation.	70
A.8. Console output, common-mode fault test: forcing two of encoder's PISO_reg[0] three redundant copies to the same wrong value. . . .	70

Generator Script and Simulation Traces

A.1 Netlist Generation Script

Section 3.5.2 describes `generate_full_security.py`'s mechanism in prose. The complete, current script is reproduced here, together with `single_counter_lib.py`, the shared module it calls into for the hierarchy-discovery and single-counter wiring described there.

The two output-path generator scripts are not reproduced separately. Both call this exact same shared function (Section 3.5.3) and differ from `generate_full_security.py` only in the `TARGET_MODULES` allow-list already shown in full there.

`generate_control_state_design.py` is the one exception; it does not call `single_counter_lib.py` at all, since `combined_top` is its only target and there is nothing to thread. It is not reproduced here either, since its logic is the same per-instance wiring pattern shown below, just without the hierarchy-threading step.

```
1  import os
2
3  from single_counter_lib import
    generate_single_counter_netlist,
    wire_trigger_rst_to_reset
4
5  REPO_ROOT    = os.path.dirname(os.path.dirname(os.path.
    abspath(__file__)))
6  NETLISTS_DIR = os.path.join(REPO_ROOT, "netlists")
7
8  input_file   = os.path.join(NETLISTS_DIR, "base_design.v")
9  output_file  = os.path.join(NETLISTS_DIR, "
    full_security_design.v")
10
11 with open(input_file, "r") as f:
12     content = f.read()
13
```

```

14 final_content, n_fault_wires, n_own_fault_modules =
    generate_single_counter_netlist(
15     content, target_modules=None
16 )
17 final_content = wire_trigger_rst_to_reset(final_content)
18
19 with open(output_file, "w") as f:
20     f.write(final_content)
21
22 print(f"Done! {n_fault_wires} fault wires generated across {
    n_own_fault_modules} protected modules.")
23 print("Single design-wide fault_counter at combined_top,
    wired into rst_IBUF.")
24 print(f"Output written to: {output_file}")

```

Listing A.1: Post-synthesis netlist transformation for full TMR protection (generate_full_security.py).

single_counter_lib.py is where the hierarchy-discovery and wiring logic actually lives. The script above only supplies the input and output paths and the target_modules scope.

```

1  import re
2
3  FF_RE = re.compile(r'\bFD(RE|SE)\b')
4  MODULE_DECL_RE = re.compile(r'^module\s+(\w+)\b')
5  INST_LINE_RE = re.compile(r'^(\s+)(\w+)\s+(\[?\s*\]\s*(?!\s*$'
6      )
7  TRM_TYPES = re.compile(r'\s+trm_fd(re|se|ce|pe)\s+#\(')
8
9  def parse_hierarchy(lines):
10      """Discover every module declaration and every real (non-
11          primitive)
12          instantiation edge in a flat list of Verilog source lines
13          .
14
15      Returns (modules, children_of, parent_of):
16          modules[name] = (start_idx, end_idx) 0-based,
17              inclusive of the
18              'module NAME' and 'endmodule'
19              lines themselves.
20          children_of[name] = [{"type", "instance", "line"},
21              ...] (line is 1-based)

```

```

17     parent_of[type]      = {"parent", "instance", "line"}
18
19     Raises if any module type is instantiated more than once
20     (this
21     netlist's modules are Vivado-uniquified per instantiation
22     site, so
23     that should never happen; if it ever does, this
24     architecture's
25     one-parent-per-module assumption needs revisiting before
26     proceeding).
27     """
28     n = len(lines)
29     modules = {}
30     i = 0
31     while i < n:
32         m = MODULE_DECL_RE.match(lines[i])
33         if m:
34             name = m.group(1)
35             start = i
36             j = i + 1
37             while j < n and lines[j].rstrip('\n') != '
38                 endmodule':
39                 j += 1
40             if j >= n:
41                 raise RuntimeError(f"module {name} (line {i
42                     +1}): no endmodule found")
43             modules[name] = (start, j)
44             i = j + 1
45         else:
46             i += 1
47
48     known = set(modules.keys())
49     children_of = {name: [] for name in modules}
50     parent_of = {}
51
52     for name, (start, end) in modules.items():
53         for k in range(start + 1, end):
54             line = lines[k].rstrip('\n')
55             if not line.strip():
56                 continue
57             mm = INST_LINE_RE.match(line)
58             if not mm:
59                 continue
60             typ, inst = mm.group(2), mm.group(3)

```

```

55         if typ in known and typ != name:
56             children_of[name].append({"type": typ, "
                instance": inst, "line": k + 1})
57         if typ in parent_of:
58             raise RuntimeError(
59                 f"module {typ} instantiated more than
                    once "
60                 f"({parent_of[typ]} and parent {name}
                    line {k+1}); "
61                 "single-counter threading assumes a
                    strict tree"
62             )
63         parent_of[typ] = {"parent": name, "instance":
                inst, "line": k + 1}
64
65     return modules, children_of, parent_of
66
67
68 def compute_needs_threading(root, children_of, own_fault_set)
69 :
70     """DFS from root: needs[name] is True if name itself has
71         its own fault
72         signal (name in own_fault_set) or any descendant does."""
73     needs = {}
74
75     def dfs(name):
76         result = name in own_fault_set
77         for kid in children_of.get(name, []):
78             if dfs(kid["type"]):
79                 result = True
80         needs[name] = result
81         return result
82
83     dfs(root)
84     return needs
85
86 def _find_header_close(out, start, end):
87     for k in range(start + 1, end):
88         if out[k].rstrip('\n').endswith(');'):
89             return k
90     raise RuntimeError(f"no header-closing ');' line found in
        range {start+1}-{end+1}")

```

```

91
92 def _add_output_port(out, modules, name, port_name):
93     start, end = modules[name]
94     k = _find_header_close(out, start, end)
95     body = out[k].rstrip('\n')
96     prefix = body[:-2] # drop the trailing ');'
97     out[k] = prefix + f",\n    {port_name});\n  output {
        port_name};\n"
98
99
100 def _find_close_paren(out, start_idx0):
101     k = start_idx0
102     while k < len(out):
103         if '));' in out[k]:
104             return k
105         k += 1
106     raise RuntimeError(f"no closing )); found starting at
        line {start_idx0+1}")
107
108
109 def _insert_port_connection(out, close_idx, port_name,
        wire_name):
110     line = out[close_idx]
111     pos = line.rindex('));')
112     out[close_idx] = line[:pos + 1] + f',\n        .{
        port_name}({wire_name});\n'
113
114
115 def generate_single_counter_netlist(content, target_modules=
        None, root='combined_top', thr_100=10):
116     """Transform base_design.v-style content into a single-
        shared-counter
117     TMR netlist.
118
119     target_modules: set of module type names whose FDRE/FDSE
        get replaced
120     with trm_fdre/trm_fdse. If None, every module
        containing at least one
121     FDRE/FDSE is targeted (the full_security_design.v case)
        .
122
123     Returns the transformed content (a single string).
124     """
125     lines = content.splitlines(keepends=True)

```

```

126
127     modules, children_of, parent_of = parse_hierarchy(lines)
128
129     if target_modules is None:
130         target_modules = {
131             name for name, (s, e) in modules.items()
132             if any(FF_RE.search(l) for l in lines[s:e + 1])
133         }
134
135     out = list(lines)
136
137     # ---- Pass 1: FDRE/FDSE -> trm_fdre/trm_fdse + per-
138         instance .fault() wiring ----
139     fault_wires_in_module = {}
140     clock_wire_in_module = {}
141
142     cur_module = None
143     in_target_module = False
144     in_trm_instance = False
145     current_fault_wire = None
146     fw_this_module = []
147     clk_this_module = None
148     global_fault_idx = 0
149
150     for idx in range(len(out)):
151         line = out[idx]
152         m_decl = MODULE_DECL_RE.match(line)
153         if m_decl:
154             cur_module = m_decl.group(1)
155             in_target_module = cur_module in target_modules
156             fw_this_module = []
157             clk_this_module = None
158             in_trm_instance = False
159
160             if in_target_module:
161                 new_line = re.sub(r'\bFDRE\b', 'trm_fdre', line)
162                 new_line = re.sub(r'\bFDSE\b', 'trm_fdse',
163                                     new_line)
164                 if new_line != line:
165                     out[idx] = new_line
166                     line = new_line
167
168             stripped = line.strip()

```

```

168         if TRM_TYPES.match(line):
169             in_trm_instance = True
170             current_fault_wire = f'fault_wire_{
                global_fault_idx}'
171             global_fault_idx += 1
172             fw_this_module.append(current_fault_wire)
173
174         if in_trm_instance and clk_this_module is None:
175             mclk = re.search(r'\.C\(((\[~])+\))\)', line)
176             if mclk:
177                 clk_this_module = mclk.group(1).strip()
178
179         if in_trm_instance and stripped.endswith('));'):
180             pos = line.rindex('));')
181             out[idx] = line[:pos + 1] + f',\n                .fault({
                current_fault_wire}));\n'
182             in_trm_instance = False
183             current_fault_wire = None
184
185         if stripped == 'endmodule' and cur_module in
            target_modules and fw_this_module:
186             fault_wires_in_module[cur_module] = list(
                fw_this_module)
187             clock_wire_in_module[cur_module] =
                clk_this_module or 'clk_IBUF_BUFG'
188
189         # ---- Determine which modules actually carry a real
            fault signal ----
190         own_fault_set = set(fault_wires_in_module.keys())
191         needs = compute_needs_threading(root, children_of,
            own_fault_set)
192
193         if not needs.get(root):
194             raise RuntimeError(
195                 "no protected flip-flops found anywhere under the
                    design root; "
196                 "nothing to count"
197             )
198
199         # ---- Pass 2a: parent-side port connections for every
            threading child ----
200         for name, ok in needs.items():
201             if not ok or name == root:
202                 continue

```

```

203     info = parent_of.get(name)
204     if info is None:
205         raise RuntimeError(f"module {name} needs
206                             threading but has no parent recorded")
207     close_idx = _find_close_paren(out, info["line"] - 1)
208     _insert_port_connection(out, close_idx, 'module_fault
209                             ', f'mf_{name}')
210     _add_output_port(out, modules, name, 'module_fault')
211
212     # ---- Pass 2b: per-module aggregation (wires + assign /
213     counter) ----
214     for name, ok in needs.items():
215         if not ok:
216             continue
217         terms = list(fault_wires_in_module.get(name, []))
218         wire_decls = [f' wire {w};' for w in
219                     fault_wires_in_module.get(name, [])]
220         for kid in children_of.get(name, []):
221             ctype = kid['type']
222             if needs.get(ctype):
223                 wname = f'mf_{ctype}'
224                 terms.append(wname)
225                 wire_decls.append(f' wire {wname};')
226
227         expr = ' | '.join(terms)
228         start, end = modules[name]
229
230         if name == root:
231             clk = clock_wire_in_module.get(name, '
232                 clk_IBUF_BUFG')
233             insertion = wire_decls + [
234                 f' (* KEEP = "true" *) wire design_any_fault
235                 = {expr};',
236                 f' (* KEEP = "true" *) wire [31:0]
237                 fault_ctr_count;',
238                 f' wire fault_ctr_trigger_rst;',
239                 f' (* DONT_TOUCH = "true" *) fault_counter
240                 #(.WIDTH(32), .THR_100({thr_100}))
241                 fault_ctr_inst ('
242                 f' .clk({clk}),',
243                 f' .rst(1'b0),",
244                 f' .fault(design_any_fault),',
245                 f' .count(fault_ctr_count),',
246                 f' .trigger_rst(fault_ctr_trigger_rst)',

```

```

238         f' );',
239     ]
240     else:
241         insertion = wire_decls + [
242             f' (* KEEP = "true" *) assign module_fault =
                {expr};',
243         ]
244
245         out[end] = '\n'.join(insertion) + '\n' + out[end]
246
247     return ''.join(out), global_fault_idx, len(own_fault_set)
248
249
250 def wire_trigger_rst_to_reset(content):
251     """Wire the single shared fault_ctr_trigger_rst (now
        always produced at
252     the root, combined_top) into rst_IBUF, same technique as
        every prior
253     per-module-counter generator script used."""
254     RST_IBUF_DRIVER = (
255         "    IBUF rst_IBUF_inst\n"
256         "        (.I(rst),\n"
257         "        .O(rst_IBUF));"
258     )
259     RST_IBUF_DRIVER_WIRED = (
260         "    wire rst_IBUF_direct;\n"
261         "    IBUF rst_IBUF_inst\n"
262         "        (.I(rst),\n"
263         "        .O(rst_IBUF_direct));\n"
264         "    assign rst_IBUF = rst_IBUF_direct |
                fault_ctr_trigger_rst;"
265     )
266     n = content.count(RST_IBUF_DRIVER)
267     if n != 1:
268         raise RuntimeError(
269             f"expected exactly one rst_IBUF driver in
                combined_top, found {n}; "
270             "netlist structure changed, update
                RST_IBUF_DRIVER before proceeding"
271         )
272     return content.replace(RST_IBUF_DRIVER,
        RST_IBUF_DRIVER_WIRED, 1)

```

Listing A.2: Shared hierarchy-discovery and single-counter wiring library (single_counter_lib.py).

Two implementation details are worth noting. The module index ranges returned by `parse_hierarchy` are computed once on the original line list, and every later edit replaces a single list element with a longer multi-line string rather than inserting or deleting elements, so those indices stay valid throughout. And every structural edit is exact-match-or-fail: `wire_trigger_rst_to_reset` counts occurrences of the expected reset-buffer pattern and raises unless there is exactly one, so a change in the netlist's structure surfaces as an error rather than as a silently incorrect transformation.

A.2 TMR Flip-Flop Wrapper: `trm_fdse`

Section 3.2.2 describes `trm_fdse` as structurally identical to `trm_fdre` (Listing 3.1) apart from a synchronous set in place of a synchronous reset, and does not reproduce it in full there for that reason. The complete module is reproduced here. Its `INIT` default is never exercised: the netlist transformation is a textual rename of `FDSE` to `trm_fdse` that leaves each instance's explicit `.INIT()` parameter in place, so every generated instance carries the initial value the original primitive was given.

```
1  module trm_fdse #(
2      parameter INIT = 1'b0
3  )(
4      output wire Q,
5      output wire fault,
6      input wire C,
7      input wire CE,
8      input wire D,
9      input wire S
10 );
11
12 (* KEEP = "true" *) wire q_a, q_b, q_c;
13
14 (* DONT_TOUCH = "true" *)
15 FDSE #(.INIT(INIT)) u_ff_a (.Q(q_a), .C(C), .CE(CE), .D(D), .
    S(S));
16 (* DONT_TOUCH = "true" *)
```

```

17  FDSE #(.INIT(INIT)) u_ff_b (.Q(q_b), .C(C), .CE(CE), .D(D), .
    S(S));
18  (* DONT_TOUCH = "true" *)
19  FDSE #(.INIT(INIT)) u_ff_c (.Q(q_c), .C(C), .CE(CE), .D(D), .
    S(S));
20
21  function vote3;
22      input a, b, c;
23      begin
24          vote3 = (a & b) | (a & c) | (b & c);
25      end
26  endfunction
27
28  (* KEEP = "true" *) wire q_voted = vote3(q_a, q_b, q_c);
29  assign Q = q_voted;
30  assign fault = (q_a ^ q_b) | (q_b ^ q_c);
31
32  endmodule

```

Listing A.3: TMR wrapper for FDSE.

A.3 Simulation Trace Evidence

Section 4.1.4 and Section 4.4 summarize the unit-level and system-level fault-injection results in prose. The raw console output and waveform evidence behind four of those claims is reproduced here.

A.3.1 Fault Counter Threshold Logic (fault_counter_tb.v)

The five threshold tests were re-run directly against the current `fault_counter.v` and `fault_counter_tb.v` via the standalone Vivado simulator (`xvlog/xelab/xsim`), with `WIDTH=32` and `THR_100=8`.¹ All five pass:

```

1  PASS [T1 no fault]
2  PASS [T2 isolated faults < THR_100]
3  PASS [T3a rolling: 7 < THR_100]
4  PASS [T3b rolling: 8 >= THR_100]
5  PASS [T4 window slide-out]

```

¹The unit test deliberately uses a lower threshold than the design default of 10 so that each case reaches it in fewer cycles. The threshold logic is independent of the specific value.

```

6  [T5] before reset: trigger_rst=1 (expected 1)
7  PASS [T5 reset clears trigger]
8  === fault_counter_tb done ===
9  $finish called at time : 2016 ns

```

Listing A.4: Console output, `fault_counter_tb.v`, all five tests passing.

The pass/fail lines above only report which side of `THR_100` the counter landed on at each checkpoint. Figure A.1 traces the internal `running_total` register directly, accessed via a hierarchical reference from a dedicated monitoring testbench rather than exposed as a port, through an equivalent scenario with `WIDTH=32`, `THR_100=8`, and a fault held for eight consecutive cycles. The rise to the threshold and the 100-cycle slide-out are both visible as the step function that the sliding-window design in Listing 3.2 predicts. `running_total` climbs by exactly one per faulty cycle. The first faulty cycle is sampled into `window[0]` at the following clock edge, $t = 15$ ns, and drops back out exactly 100 cycles later at $t = 15 + 100 \times 10 = 1015$ ns. That single edge both starts the count falling and takes `running_total` below the threshold, so `trigger_rst` clears on the same edge rather than afterwards.

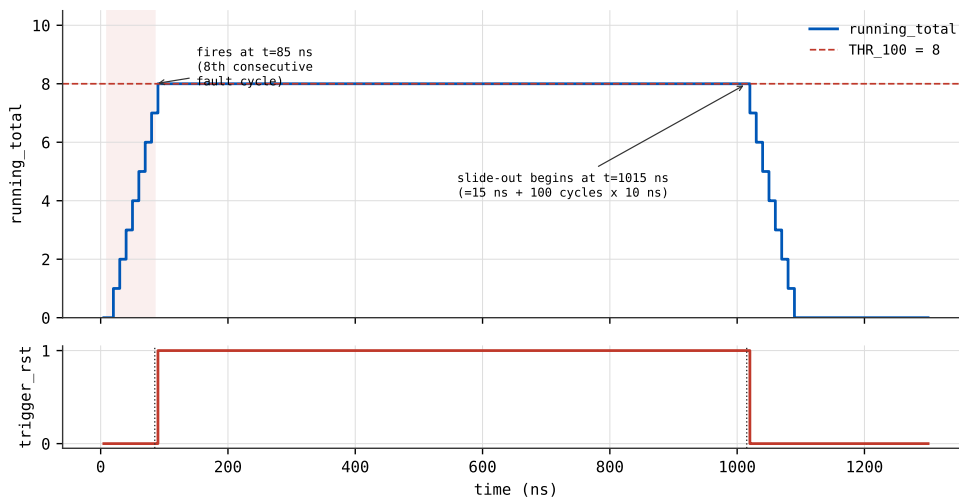


Fig. A.1.: Fault counter sliding window: rise to `THR_100=8` and the subsequent 100-cycle slide-out, traced from the internal `running_total` register.

A.3.2 TMR Masking Guarantee, Traced Directly (based on `fdre_tb.v`)

A `$monitor`-instrumented variant of `fdre_tb.v` prints every signal transition of `D`, `Q`, the three internal copies `q_a/q_b/q_c`, `fault`, and `trigger_rst` throughout the

run. The excerpt below covers the fault-injection window: a stuck-at-1 forced onto `q_a` at 500 ns, switched to stuck-at-0 at 600 ns, and released at 700 ns, against a `THR_100=10` counter.

```

1  t=495000 D=1 Q=1 q_a=1 q_b=1 q_c=1 fault=0 trig=0
2  [500000] injecting stuck-at-1 on q_a | trigger_rst = 0
3  t=500000 D=0 Q=1 q_a=1 q_b=1 q_c=1 fault=0 trig=0
4  t=505000 D=0 Q=0 q_a=1 q_b=0 q_c=0 fault=1 trig=0
5  ...
6  [600000] switching to stuck-at-0 on q_a | trigger_rst = 0
7  t=600000 D=0 Q=1 q_a=0 q_b=1 q_c=1 fault=1 trig=0
8  ...
9  t=680000 D=0 Q=1 q_a=0 q_b=1 q_c=1 fault=1 trig=0
10 t=685000 D=0 Q=1 q_a=0 q_b=1 q_c=1 fault=1 trig=1
11 t=685000 D=0 Q=0 q_a=0 q_b=0 q_c=0 fault=0 trig=1
12 [700000] fault released | trigger_rst = 1
13 t=700000 D=0 Q=1 q_a=1 q_b=1 q_c=1 fault=0 trig=1
14 ...
15 [5000000] === final trigger_rst: 0 ===

```

Listing A.5: \$monitor trace excerpt around fault injection, full run is 1000 lines over 5 μ s.

The excerpt establishes the masking guarantee directly rather than by argument: `Q` never once follows the forced `q_a`, neither during the stuck-at-1 phase, where it continues tracking `q_b/q_c` whenever they disagree with `q_a`, nor during the stuck-at-0 phase. It also shows `trigger_rst` asserting at 685 ns, once the sliding-window running total (Listing 3.2) reaches `THR_100=10`.

Figure A.2 plots the same run as a digital waveform rather than a raw signal dump, which makes the clearing point visible as well. `trigger_rst` de-asserts at 1515 ns, and that point follows from the window arithmetic rather than from when the injection ended. The episode's *first* fault-active cycle, sampled at 505 ns, enters `window[0]` at the next edge, 515 ns, the same one-cycle sampling delay noted in Section A.3.1, and leaves the 100-cycle window exactly 100 cycles later, at 1515 ns. `running_total` and the registered `trigger_rst` both update from that departure on that same edge, dropping the running total to 9 and clearing the trigger together. This is the same-edge behavior established in Section A.3.1, not a second cycle of delay on top of it.

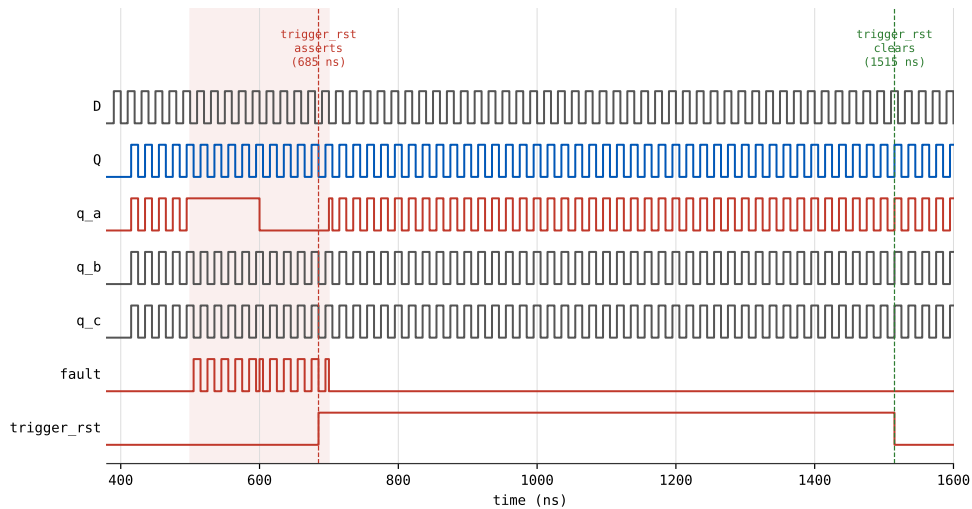


Fig. A.2.: Digital waveform of the \$monitor trace in Listing A.5: Q tracks the majority value throughout the stuck-at-1 (500–600 ns) and stuck-at-0 (600–700 ns) injection, while trigger_rst asserts at 685 ns and clears at 1515 ns.

A.3.3 Fault-Injection Testbench, Configuration and Force Mechanism

rtl_tb/tb_sign_trmff_fault_injection_top.v drives every system-level case in Section 4.4. A case is selected by editing a block of ‘define macros at the top of the file rather than by passing arguments, so the specific macro state a given run used is part of that run’s record. The excerpt below shows the macro block left after the common-mode case (case 6) together with the force branch it selects, in which both q_a and q_b of a protected register are forced to the same value for one phase and then to the other. This is the mechanism behind Listing A.8. The full file is 812 lines; only the configuration block and this one force branch are shown.

```

1  'define TARGET_D
2  'define PROTECTED
3  'define COMMON_MODE
4  'define COMMON_MODE_OFFSET_NS 83600
5  'define P 10
6  ...
7  'ifdef TARGET_D
8      'ifdef PROTECTED
9          'ifdef COMMON_MODE
10             // Forces q_a AND q_b together instead of only q_a:
                one physical

```

```

11      // event corrupting two of three redundant copies at
      once, which
12      // vote3's plain majority function cannot distinguish
      from a
13      // genuine 2-of-3 agreement.
14      #( 'COMMON_MODE_OFFSET_NS);
15      $display("[%0t] TARGET_D (protected, COMMON_MODE,
      offset=%0dns, phase 1/2): forcing DUT.ENCODER.\
      PISO_reg[0] .q_a AND .q_b = 1", $time, '
      COMMON_MODE_OFFSET_NS);
16      force DUT.ENCODER.\PISO_reg[0] .q_a = 1'b1;
17      force DUT.ENCODER.\PISO_reg[0] .q_b = 1'b1;
18      #40;
19      $display("[%0t] TARGET_D COMMON_MODE phase 2/2:
      forcing q_a/q_b = 0", $time);
20      force DUT.ENCODER.\PISO_reg[0] .q_a = 1'b0;
21      force DUT.ENCODER.\PISO_reg[0] .q_b = 1'b0;
22      #40;
23      release DUT.ENCODER.\PISO_reg[0] .q_a;
24      release DUT.ENCODER.\PISO_reg[0] .q_b;
25      'else
26      ...

```

Listing A.6: Case selection macros and the common-mode force branch.

The two-phase force (1'b1 then 1'b0) rather than a single stuck value is deliberate. A single value only creates an observable fault if it happens to disagree with the correct value for the specific KAT vector under test, which for a single bit is an even chance, whereas two opposite phases guarantee that at least one phase disagrees regardless of what the correct value turns out to be. Every other target and case combination in Table 4.9 follows the same pattern, a 'define block selecting one 'ifdef branch, with only the target signal, the timing offset, and the choice of single-versus common-mode force differing.

A.3.4 System-Level Fault Injection, Two Illustrative Cases

Section 4.4 summarizes all six system-level fault-injection cases in prose. Two results are reproduced here, chosen to illustrate two distinct kinds of evidence the campaign provides: the fault counter's reset mechanism firing end-to-end inside a real signing operation, and a confirmed common-mode corruption. The TARGET_ labels in the traces below are the testbench's own switches for which register to force. TARGET_C

is `ctr fsm1`, used here for the reset-mechanism test, which carries no case number of its own, and `TARGET_D` is encoder's `PISO_reg`, used here for case 6.

```

1  [500000] TARGET_C (protected, sustained, phase 1/2): forcing
    DUT.\ctr fsm1_reg[0].q_a = 1
2  [590000] DUT.fault_ctr_trigger_rst -> 1, DUT.rst_IBUF -> 1
3  [650000] TARGET_C phase 2/2: forcing = 0
4  [800000] fault released
5  [1550000] DUT.fault_ctr_trigger_rst -> 0, DUT.rst_IBUF -> 0

```

Listing A.7: Console output, reset-mechanism test: `fault_ctr_trigger_rst/rst_IBUF` asserting and clearing inside a real signing operation.

A sustained fault forced onto a protected `ctr fsm1` register crosses `THR_100` at the tenth faulty edge. The ten faulty edges fall at $t = 500, 510, \dots, 590$ ns, with the force beginning exactly on the first of them, so nine periods, 90 ns, separate the first edge from the tenth, both endpoints counted. The crossing at $t = 590$ ns asserts `fault_ctr_trigger_rst` and `rst_IBUF` together, and both clear together at $t = 1550$ ns once the sliding window ages the fault back out. This is the complete assert-then-clear lifecycle observed directly inside a real signing operation, previously demonstrated only in the isolated `fault_counter_tb.v` unit test (Section A.3.1).

```

1  [104380000] TARGET_D (protected, COMMON_MODE, offset=83600ns,
    phase 1/2): forcing DUT.ENCODER.\PISO_reg[0].q_a AND .
    q_b = 1
2  [104420000] TARGET_D COMMON_MODE phase 2/2: forcing q_a/q_b =
    0
3  [Z, 1] Error: Expected 73d7c8fc5662f207, received 72
    d7c8fc5662f207
4  [Z, 2] Error: Expected bf58ea381cd4a3a0, received
    be58ea381cd4a3a0
5  [104460000] fault released
6  VERDICT: CORRUPTED (signature wrong, 2 mismatches)

```

Listing A.8: Console output, common-mode fault test: forcing two of encoder's `PISO_reg[0]` three redundant copies to the same wrong value.

Forcing `q_a` and `q_b` to the same incorrect value simultaneously, at an offset independently confirmed to overlap this register's real read cycle, produces two mismatched output words at the same bit position an unprotected calibration run also mismatched, which rules out coincidence. The 83,600 ns offset in the trace is the injection delay applied by the testbench, a round number chosen to place the window just before the first transfer at $t = 104,410$ ns; it is not the same quantity as the

~83,630 ns gap between that transfer and the assumed $t = 20,780$ ns discussed in Section 4.4. `vote3`'s plain majority function has no way to distinguish two corrupted copies from a genuine 2-of-3 majority, so this directly confirms TMR's common-mode blind spot rather than only arguing for it.